```
cloudra/__init__.py:1   """Cloudra — AI Agent collaboration framework for chemical research."""
cloudra/__init__.py:2
cloudra/__init__.py:3   from importlib import import_module
cloudra/__init__.py:4
cloudra/__init__.py:5   __version__ = "2.0.0-beta"
cloudra/__init__.py:6
cloudra/__init__.py:7   _LAZY_EXPORTS = {
cloudra/__init__.py:8       "Agent": ("cloudra.core.agent", "Agent"),
cloudra/__init__.py:9       "Coordinator": ("cloudra.core.agent", "Coordinator"),
cloudra/__init__.py:10  }
cloudra/__init__.py:11  # 兼容垫片：包结构重排（core/services/persistence/base/infra）前的
cloudra/__init__.py:12  # `from cloudra import X` 老路径全部继续可用（惰性加载，不提前拉起重依赖）。
cloudra/__init__.py:13  _LAZY_MODULES = {
cloudra/__init__.py:14      # core
cloudra/__init__.py:15      "agent": "cloudra.core.agent",
cloudra/__init__.py:16      # services
cloudra/__init__.py:17      "session_store": "cloudra.services.session_store",
cloudra/__init__.py:18      "nodes": "cloudra.services.nodes",
cloudra/__init__.py:19      # persistence
cloudra/__init__.py:20      "database": "cloudra.persistence.database",
cloudra/__init__.py:21      "artifacts": "cloudra.persistence.artifacts",
cloudra/__init__.py:22      "memory": "cloudra.persistence.memory",
cloudra/__init__.py:23      "message_history": "cloudra.core.message_history",
cloudra/__init__.py:24      "paths": "cloudra.persistence.paths",
cloudra/__init__.py:25      "fs_utils": "cloudra.persistence.fs_utils",
cloudra/__init__.py:26      "workspace": "cloudra.persistence.workspace",
cloudra/__init__.py:27      # base
cloudra/__init__.py:28      "session_class": "cloudra.base.session_class",
cloudra/__init__.py:29      "skill_class": "cloudra.base.skill_class",
cloudra/__init__.py:30      "tools_class": "cloudra.base.tools_class",
cloudra/__init__.py:31      # infra
cloudra/__init__.py:32      "config": "cloudra.infra.config",
cloudra/__init__.py:33      "llm_client": "cloudra.infra.llm_client",
cloudra/__init__.py:34      "streaming": "cloudra.infra.streaming",
cloudra/__init__.py:35      "token_estimator": "cloudra.infra.token_estimator",
cloudra/__init__.py:36      "web_search_native": "cloudra.infra.web_search_native",
cloudra/__init__.py:37      "slurm": "cloudra.infra.slurm",
cloudra/__init__.py:38      "compute_utils": "cloudra.infra.compute_utils",
cloudra/__init__.py:39      "operations": "cloudra.infra.operations",
cloudra/__init__.py:40      "yolo_guard": "cloudra.infra.yolo_guard",
cloudra/__init__.py:41      "platform_utils": "cloudra.infra.platform_utils",
cloudra/__init__.py:42      "runtime_profile": "cloudra.infra.runtime_profile",
cloudra/__init__.py:43  }
cloudra/__init__.py:44
cloudra/__init__.py:45  __all__ = [*_LAZY_EXPORTS, *_LAZY_MODULES, "__version__"]
cloudra/__init__.py:46
cloudra/__init__.py:47
cloudra/__init__.py:48  def __getattr__(name: str):
cloudra/__init__.py:49      """Keep legacy package exports without importing the Agent runtime eagerly."""
cloudra/__init__.py:50      target = _LAZY_EXPORTS.get(name)
cloudra/__init__.py:51      if target is not None:
cloudra/__init__.py:52          module_name, attribute = target
```

```
cloudra/__init__.py:53          value = getattr(import_module(module_name), attribute)
cloudra/__init__.py:54          globals()[name] = value
cloudra/__init__.py:55          return value
cloudra/__init__.py:56      module_path = _LAZY_MODULES.get(name)
cloudra/__init__.py:57      if module_path is not None:
cloudra/__init__.py:58          value = import_module(module_path)
cloudra/__init__.py:59          globals()[name] = value
cloudra/__init__.py:60          return value
cloudra/__init__.py:61      raise AttributeError(f"module 'cloudra' has no attribute {name!r}")
cloudra/__init__.py:62
cloudra/__init__.py:63
cloudra/__init__.py:64  def __dir__() -> list[str]:
cloudra/__init__.py:65      return sorted(set(globals()) | set(__all__))
cloudra/__main__.py:1   """Allow running as `python -m cloudra`."""
cloudra/__main__.py:2   from cloudra.cli import main
cloudra/__main__.py:3
cloudra/__main__.py:4   if __name__ == "__main__":
cloudra/__main__.py:5       main()
cloudra/base/__init__.py:1  """cloudra.base 子包。"""
cloudra/base/session_class.py:1
cloudra/base/session_class.py:2 import os
cloudra/base/session_class.py:3 from datetime import datetime
cloudra/base/session_class.py:4
cloudra/base/session_class.py:5 from cloudra.persistence.fs_utils import atomic_write_json
cloudra/base/session_class.py:6
cloudra/base/session_class.py:7
cloudra/base/session_class.py:8 class Session:
cloudra/base/session_class.py:9     """内存态会话镜像；数据库（session 表）是唯一持久化事实来源。
cloudra/base/session_class.py:10
cloudra/base/session_class.py:11        默认不再写 per-session JSON 快照文件；调试时可设置环境变量
cloudra/base/session_class.py:12        ``CLOUDRA_DEBUG_SESSION_JSON=1`` 恢复写文件行为（upload_json 仍可显式调用）。
cloudra/base/session_class.py:13        """
cloudra/base/session_class.py:14        id: str
cloudra/base/session_class.py:15        path: str
cloudra/base/session_class.py:16        create_time: datetime
cloudra/base/session_class.py:17        state: str = 'sleeping'
cloudra/base/session_class.py:18
cloudra/base/session_class.py:19        def __init__(self, id: str, path: str) -> None:
cloudra/base/session_class.py:20            self.create_time = datetime.now()
cloudra/base/session_class.py:21            self.id = id
cloudra/base/session_class.py:22            self.path = path
cloudra/base/session_class.py:23            self.state = 'sleeping'
cloudra/base/session_class.py:24            if os.environ.get("CLOUDRA_DEBUG_SESSION_JSON") == "1":
cloudra/base/session_class.py:25                dir_name = os.path.dirname(self.path)
cloudra/base/session_class.py:26                if dir_name:
cloudra/base/session_class.py:27                    os.makedirs(dir_name, exist_ok=True)
cloudra/base/session_class.py:28                self.upload_json()
cloudra/base/session_class.py:29
cloudra/base/session_class.py:30
cloudra/base/session_class.py:31        def to_json(self) -> dict:
cloudra/base/session_class.py:32            create_time = self.create_time
cloudra/base/session_class.py:33            if isinstance(create_time, datetime):
```

```
cloudra/base/session_class.py:34                create_time = create_time.isoformat()
cloudra/base/session_class.py:35            json_dict = {
cloudra/base/session_class.py:36                'id': self.id,
cloudra/base/session_class.py:37                'path': self.path,
cloudra/base/session_class.py:38                'create_time': create_time,
cloudra/base/session_class.py:39                'state': self.state,
cloudra/base/session_class.py:40            }
cloudra/base/session_class.py:41            return json_dict
cloudra/base/session_class.py:42
cloudra/base/session_class.py:43        def upload_json(self):
cloudra/base/session_class.py:44            atomic_write_json(self.path, self.to_json())
cloudra/base/session_class.py:45
cloudra/base/skill_class.py:1   import hashlib
cloudra/base/skill_class.py:2   import os
cloudra/base/skill_class.py:3
cloudra/base/skill_class.py:4   import frontmatter as frt
cloudra/base/skill_class.py:5
cloudra/base/skill_class.py:6   # 模块级缓存按 SKILL.md 内容 revision 失效。Phase 6 后 skill manifest
cloudra/base/skill_class.py:7   # 允许在进程运行期间更新，不能再假设目录在整个进程生命周期中静态不变。
cloudra/base/skill_class.py:8   _SKILL_DATAS_CACHE: dict[str, tuple[str, list[dict]]] = {}
cloudra/base/skill_class.py:9
cloudra/base/skill_class.py:10  # NOTE: Skills 模型仅作为 skill frontmatter 的字段参照，当前未被实例化使用。
cloudra/base/skill_class.py:11  # 若后续需要校验 skill 元数据，可取消注释并补全默认值。
cloudra/base/skill_class.py:12  # class Skills(BaseModel):
cloudra/base/skill_class.py:13  #     name: str
cloudra/base/skill_class.py:14  #     description: str
cloudra/base/skill_class.py:15  #     path: str
cloudra/base/skill_class.py:16  #     scripts: list = []
cloudra/base/skill_class.py:17  #     references: list = []
cloudra/base/skill_class.py:18
cloudra/base/skill_class.py:19
cloudra/base/skill_class.py:20  class skill_loader:
cloudra/base/skill_class.py:21
cloudra/base/skill_class.py:22      # Skills are bundled with the package, not user data（base/ 上溯一级到包根）
cloudra/base/skill_class.py:23      base_path: str = os.path.join(
cloudra/base/skill_class.py:24          os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'skills'
cloudra/base/skill_class.py:25      )
cloudra/base/skill_class.py:26
cloudra/base/skill_class.py:27      def __init__(self):
cloudra/base/skill_class.py:28          # FIX: 原代码直接用了模块级变量 base_path（缺少 self.），在实例方法中访问类属性应加 self
cloudra/base/skill_class.py:29          self.namelst = os.listdir(self.base_path)
cloudra/base/skill_class.py:30
cloudra/base/skill_class.py:31          self.pathlst = [os.path.join(self.base_path, skill_name) for skill_name in self.
namelst]
cloudra/base/skill_class.py:32
cloudra/base/skill_class.py:33          self.md_pathlst = [os.path.join(path,'SKILL.md') for path in self.pathlst if os.
path.isdir(path)]
cloudra/base/skill_class.py:34
cloudra/base/skill_class.py:35      def skill_datas_get(self) -> list[dict]:
cloudra/base/skill_class.py:36          revision_hash = hashlib.sha256()
cloudra/base/skill_class.py:37          for md in sorted(self.md_pathlst):
cloudra/base/skill_class.py:38              revision_hash.update(md.encode("utf-8"))
cloudra/base/skill_class.py:39              try:
cloudra/base/skill_class.py:40                  with open(md, "rb") as stream:
```

```
cloudra/base/skill_class.py:41                    revision_hash.update(stream.read())
cloudra/base/skill_class.py:42                except OSError:
cloudra/base/skill_class.py:43                    continue
cloudra/base/skill_class.py:44            revision = revision_hash.hexdigest()
cloudra/base/skill_class.py:45            cached = _SKILL_DATAS_CACHE.get(self.base_path)
cloudra/base/skill_class.py:46            if cached is None or cached[0] != revision:
cloudra/base/skill_class.py:47                mds = self.md_pathlst
cloudra/base/skill_class.py:48                datas = []
cloudra/base/skill_class.py:49                for md in mds:
cloudra/base/skill_class.py:50                    with open(md, 'r', encoding='utf-8') as f:
cloudra/base/skill_class.py:51                        post = frt.load(f)
cloudra/base/skill_class.py:52                        metadata = post.metadata
cloudra/base/skill_class.py:53                        content = post.content
cloudra/base/skill_class.py:54
cloudra/base/skill_class.py:55                        # FIX: dict.update() 返回 None，原代码把 None 当数据 append 了
cloudra/base/skill_class.py:56                        # 这里直接修改 metadata dict 后 append（frontmatter 的 metadata 是普通 dict
）
cloudra/base/skill_class.py:57                        metadata['content'] = content
cloudra/base/skill_class.py:58                        datas.append(metadata)
cloudra/base/skill_class.py:59
cloudra/base/skill_class.py:60                _SKILL_DATAS_CACHE[self.base_path] = (revision, datas)
cloudra/base/skill_class.py:61
cloudra/base/skill_class.py:62            return _SKILL_DATAS_CACHE[self.base_path][1]
cloudra/base/skill_class.py:63
cloudra/base/skill_class.py:64        def get_reference(self, skill_name, ref_name) -> str:
cloudra/base/skill_class.py:65            # 支持多种可能的资料文件夹命名约定
cloudra/base/skill_class.py:66            possible_dirs = ['references', 'reference', 'ref']
cloudra/base/skill_class.py:67
cloudra/base/skill_class.py:68            for dir_name in possible_dirs:
cloudra/base/skill_class.py:69                ref_path = os.path.join(self.base_path, skill_name, dir_name, ref_name)
cloudra/base/skill_class.py:70                if os.path.isfile(ref_path):
cloudra/base/skill_class.py:71                    with open(ref_path, 'r', encoding='utf-8') as f:
cloudra/base/skill_class.py:72                        return f.read()
cloudra/base/skill_class.py:73                elif os.path.isdir(ref_path):
cloudra/base/skill_class.py:74                    dir_filelst = os.listdir(ref_path)
cloudra/base/skill_class.py:75                    return fr'该reference为文件夹，文件夹下有：{dir_filelst} ,请根据需要输入 {ref_name
}/filename 调用资料，win系统请用 {ref_name}\filename'
cloudra/base/skill_class.py:76
cloudra/base/skill_class.py:77            return '该资料不存在。'
cloudra/base/skill_class.py:78
cloudra/base/skill_class.py:79        # FIX: 原拼写为 skill_dsatas_get（多打了个 s）
cloudra/base/skill_class.py:80        def get_content(self, skill_name) -> str:
cloudra/base/skill_class.py:81            datas = self.skill_datas_get()
cloudra/base/skill_class.py:82            for data in datas:
cloudra/base/skill_class.py:83                if data['name'] == skill_name:
cloudra/base/skill_class.py:84                    return data['content']
cloudra/base/skill_class.py:85            return 'skill不存在。'
cloudra/base/skill_class.py:86
cloudra/base/skill_class.py:87
cloudra/base/skill_class.py:88
cloudra/base/skill_class.py:89
cloudra/base/skill_class.py:90
cloudra/base/skill_class.py:91
cloudra/base/skill_class.py:92
```

```
cloudra/base/skill_class.py:93
cloudra/base/skill_class.py:94
cloudra/base/skill_class.py:95
cloudra/base/skill_class.py:96
cloudra/base/skill_class.py:97
cloudra/base/skill_class.py:98
cloudra/base/skill_class.py:99
cloudra/base/tools_class.py:1   from contextlib import contextmanager
cloudra/base/tools_class.py:2   from contextvars import ContextVar
cloudra/base/tools_class.py:3   from typing import TYPE_CHECKING, Any, Awaitable, Callable, Optional
cloudra/base/tools_class.py:4
cloudra/base/tools_class.py:5   from pydantic import BaseModel
cloudra/base/tools_class.py:6   from pydantic_core import PydanticUndefined
cloudra/base/tools_class.py:7
cloudra/base/tools_class.py:8   from cloudra.base.session_class import Session
cloudra/base/tools_class.py:9   from cloudra.persistence.database import ConversationDB, get_database_path
cloudra/base/tools_class.py:10
cloudra/base/tools_class.py:11  # HACK: 这里原本缺少 import，属于历史遗留问题；get_tools_lst 定义在 tools/__init__.py
cloudra/base/tools_class.py:12  from cloudra.tools import get_tools_lst
cloudra/base/tools_class.py:13  from cloudra.tools.core.permission_model import Permission
cloudra/base/tools_class.py:14
cloudra/base/tools_class.py:15  if TYPE_CHECKING:
cloudra/base/tools_class.py:16      from cloudra.infra.yolo_guard import YoloGuard
cloudra/base/tools_class.py:17
cloudra/base/tools_class.py:18  # runtime_v2 的审批钩子：LegacyToolExecutor 在执行工具前总是设置；
cloudra/base/tools_class.py:19  # 未设置即生产外路径（v1/CLI/Flet 已删除）、request_approval 直接抛错。
cloudra/base/tools_class.py:20  # 签名：async (**fields) -> tuple[bool, str]，字段同 request_approval 的关键字参数。
cloudra/base/tools_class.py:21  _APPROVAL_HANDLER: ContextVar[
cloudra/base/tools_class.py:22      Optional[Callable[..., Awaitable[tuple[bool, str]]]]
cloudra/base/tools_class.py:23  ] = ContextVar("cloudra_approval_handler", default=None)
cloudra/base/tools_class.py:24
cloudra/base/tools_class.py:25
cloudra/base/tools_class.py:26  @contextmanager
cloudra/base/tools_class.py:27  def use_approval_handler(handler: Callable[..., Awaitable[tuple[bool, str]]]):
cloudra/base/tools_class.py:28      """Scope a v2 approval handler to one tool execution (per-task isolation)."""
cloudra/base/tools_class.py:29      token = _APPROVAL_HANDLER.set(handler)
cloudra/base/tools_class.py:30      try:
cloudra/base/tools_class.py:31          yield
cloudra/base/tools_class.py:32      finally:
cloudra/base/tools_class.py:33          _APPROVAL_HANDLER.reset(token)
cloudra/base/tools_class.py:34
cloudra/base/tools_class.py:35
cloudra/base/tools_class.py:36  def _tool_db_path(subagent) -> str:
cloudra/base/tools_class.py:37      """根据 tool 关联的 Agent 返回对应 conversation 的数据库路径。"""
cloudra/base/tools_class.py:38      conv_id = getattr(subagent, 'conversation_id', 'default') if subagent else 'default'
cloudra/base/tools_class.py:39      return get_database_path(conv_id)
cloudra/base/tools_class.py:40
cloudra/base/tools_class.py:41
cloudra/base/tools_class.py:42  def get_tools() -> list[dict]:
cloudra/base/tools_class.py:43      """将工具类加载为 OpenAI function 格式的 dict 列表。"""
cloudra/base/tools_class.py:44      entries = get_tools_lst()
cloudra/base/tools_class.py:45      tools_list = []
```

```
cloudra/base/tools_class.py:46      for entry in entries:
cloudra/base/tools_class.py:47          tool_instance = entry.instantiate()
cloudra/base/tools_class.py:48          tools_list.append(tool_instance.tool_schema_json())
cloudra/base/tools_class.py:49      return tools_list
cloudra/base/tools_class.py:50
cloudra/base/tools_class.py:51
cloudra/base/tools_class.py:52  class input_schema(BaseModel):
cloudra/base/tools_class.py:53      '''
cloudra/base/tools_class.py:54      工具输入模式基类，定义工具需要的输入参数和格式。
cloudra/base/tools_class.py:55
cloudra/base/tools_class.py:56      子类用法示例：
cloudra/base/tools_class.py:57          class MyInput(input_schema):
cloudra/base/tools_class.py:58              query: str = Field(description="搜索关键词")
cloudra/base/tools_class.py:59              max_results: int = Field(default=10, description="最大返回条数")
cloudra/base/tools_class.py:60
cloudra/base/tools_class.py:61      通过 to_parameters_schema() 可自动生成符合 OpenAI Function Calling 规范的 parameters 字典。
cloudra/base/tools_class.py:62      '''
cloudra/base/tools_class.py:63
cloudra/base/tools_class.py:64      @classmethod
cloudra/base/tools_class.py:65      def to_parameters_schema(cls) -> dict:
cloudra/base/tools_class.py:66          '''
cloudra/base/tools_class.py:67          生成符合 OpenAI Function Calling 的 parameters 字典。
cloudra/base/tools_class.py:68          基于 Pydantic model_json_schema() 微调：
cloudra/base/tools_class.py:69          - properties / required 已由 Pydantic 根据字段声明自动生成
cloudra/base/tools_class.py:70          - 强制附加 additionalProperties: false，防止 LLM 传入未定义字段
cloudra/base/tools_class.py:71          '''
cloudra/base/tools_class.py:72          schema = cls.model_json_schema()
cloudra/base/tools_class.py:73          parameters = {
cloudra/base/tools_class.py:74              "type": "object",
cloudra/base/tools_class.py:75              "properties": schema.get("properties", {}),
cloudra/base/tools_class.py:76              "required": schema.get("required", []),
cloudra/base/tools_class.py:77              "additionalProperties": False,
cloudra/base/tools_class.py:78          }
cloudra/base/tools_class.py:79          if "$defs" in schema:
cloudra/base/tools_class.py:80              parameters["$defs"] = schema["$defs"]
cloudra/base/tools_class.py:81          return parameters
cloudra/base/tools_class.py:82
cloudra/base/tools_class.py:83      @classmethod
cloudra/base/tools_class.py:84      def from_args(cls, arg_dict: dict) -> "input_schema":
cloudra/base/tools_class.py:85          '''从 LLM 返回的 arguments dict 构造输入实例，带 Pydantic 类型校验。'''
cloudra/base/tools_class.py:86          return cls(**arg_dict)
cloudra/base/tools_class.py:87
cloudra/base/tools_class.py:88      def toClass(self, arg_dict: dict) -> "input_schema":
cloudra/base/tools_class.py:89          '''
cloudra/base/tools_class.py:90          兼容旧接口，供 reAct.py 调用。
cloudra/base/tools_class.py:91          实际调用链通常为 tool.input_schema.toClass(tool_args)，此时 self 为 input_schema 子类本
身。
cloudra/base/tools_class.py:92          '''
cloudra/base/tools_class.py:93          # 当通过类调用时（如 TextReader_input_schema.toClass(args)），self 即为类对象
cloudra/base/tools_class.py:94          if isinstance(self, type) and issubclass(self, input_schema):
cloudra/base/tools_class.py:95              return self(**arg_dict)
cloudra/base/tools_class.py:96          return self.__class__(**arg_dict)
cloudra/base/tools_class.py:97
```

```
cloudra/base/tools_class.py:98      def toJson(self) -> dict:
cloudra/base/tools_class.py:99          '''实例序列化为 dict，用于日志或上下文回传。'''
cloudra/base/tools_class.py:100         return self.model_dump(mode="json")
cloudra/base/tools_class.py:101
cloudra/base/tools_class.py:102
cloudra/base/tools_class.py:103 class output_schema(BaseModel):
cloudra/base/tools_class.py:104     '''
cloudra/base/tools_class.py:105     工具输出模式基类，定义工具输出的格式。
cloudra/base/tools_class.py:106     子类按普通 Pydantic Model 声明字段即可。
cloudra/base/tools_class.py:107     '''
cloudra/base/tools_class.py:108
cloudra/base/tools_class.py:109     def toJson(self) -> dict:
cloudra/base/tools_class.py:110         return self.model_dump(mode="json")
cloudra/base/tools_class.py:111
cloudra/base/tools_class.py:112
cloudra/base/tools_class.py:113 class Tool(BaseModel):
cloudra/base/tools_class.py:114     '''
cloudra/base/tools_class.py:115     工具基类。所有具体工具应继承此类。
cloudra/base/tools_class.py:116
cloudra/base/tools_class.py:117     关于 input_schema / output_schema：
cloudra/base/tools_class.py:118     - 这里存放的是具体的 schema 类（如 TextReader_input_schema），而非实例。
cloudra/base/tools_class.py:119     - 使用 Any 类型并开启 arbitrary_types_allowed，避免 Pydantic 对类对象做字段校验。
cloudra/base/tools_class.py:120     - 子模块中覆盖时必须带类型注解，例如：
cloudra/base/tools_class.py:121           input_schema: Any = TextReader_input_schema
cloudra/base/tools_class.py:122     '''
cloudra/base/tools_class.py:123     model_config = {"arbitrary_types_allowed": True}
cloudra/base/tools_class.py:124
cloudra/base/tools_class.py:125     # 唯一标识符
cloudra/base/tools_class.py:126     tool_id: str = ''
cloudra/base/tools_class.py:127
cloudra/base/tools_class.py:128     # 工具名称（即 OpenAI function.name）
cloudra/base/tools_class.py:129     tool_name: str = ''
cloudra/base/tools_class.py:130
cloudra/base/tools_class.py:131     # 工具描述。改造后应仅保留【功能描述】+【使用场景】，不再包含参数细节。
cloudra/base/tools_class.py:132     # 参数说明通过 input_schema 的 Field(description="...") 提供，自动生成到 parameters 中。
cloudra/base/tools_class.py:133     tool_description: str = ''
cloudra/base/tools_class.py:134
cloudra/base/tools_class.py:135     # 工具的输入模式类（存放类对象，非实例）
cloudra/base/tools_class.py:136     input_schema: Any = input_schema
cloudra/base/tools_class.py:137
cloudra/base/tools_class.py:138     # 工具的输出模式类（存放类对象，非实例）
cloudra/base/tools_class.py:139     output_schema: Any = output_schema
cloudra/base/tools_class.py:140
cloudra/base/tools_class.py:141     # 引用调用该工具的 Agent 实例
cloudra/base/tools_class.py:142     subagent: Any = None
cloudra/base/tools_class.py:143
cloudra/base/tools_class.py:144     def is_read(self) -> bool:
cloudra/base/tools_class.py:145         return False
cloudra/base/tools_class.py:146
cloudra/base/tools_class.py:147     def is_write(self) -> bool:
cloudra/base/tools_class.py:148         return False
cloudra/base/tools_class.py:149
```

```
cloudra/base/tools_class.py:150     def is_manage(self) -> bool:
cloudra/base/tools_class.py:151         return False
cloudra/base/tools_class.py:152
cloudra/base/tools_class.py:153     def is_script(self) -> bool:
cloudra/base/tools_class.py:154         return False
cloudra/base/tools_class.py:155
cloudra/base/tools_class.py:156     def is_destructive(self) -> bool:
cloudra/base/tools_class.py:157         return False
cloudra/base/tools_class.py:158
cloudra/base/tools_class.py:159     def tool_permission_all(self) -> Permission:
cloudra/base/tools_class.py:160         """Return Permission flags for this tool."""
cloudra/base/tools_class.py:161         perms = Permission.NONE
cloudra/base/tools_class.py:162         if self.is_read():
cloudra/base/tools_class.py:163             perms |= Permission.READ
cloudra/base/tools_class.py:164         if self.is_write():
cloudra/base/tools_class.py:165             perms |= Permission.WRITE
cloudra/base/tools_class.py:166         if self.is_manage():
cloudra/base/tools_class.py:167             perms |= Permission.MANAGE
cloudra/base/tools_class.py:168         if self.is_script():
cloudra/base/tools_class.py:169             perms |= Permission.SCRIPT
cloudra/base/tools_class.py:170         if self.is_destructive():
cloudra/base/tools_class.py:171             perms |= Permission.DESTRUCTIVE
cloudra/base/tools_class.py:172         return perms
cloudra/base/tools_class.py:173
cloudra/base/tools_class.py:174     async def tool_call(self, user_input: input_schema, session: Session) -> output_schema:
cloudra/base/tools_class.py:175         '''
cloudra/base/tools_class.py:176         调用工具。子类必须重写此方法。
cloudra/base/tools_class.py:177         执行结果以 output_schema 实例返回。
cloudra/base/tools_class.py:178         '''
cloudra/base/tools_class.py:179         raise NotImplementedError("子类必须实现 tool_call 方法")
cloudra/base/tools_class.py:180
cloudra/base/tools_class.py:181     async def safe_call(self, user_input: input_schema, session: Session) -> output_schema:
cloudra/base/tools_class.py:182         '''
cloudra/base/tools_class.py:183         带有通用异常捕获的工具调用包装器。
cloudra/base/tools_class.py:184         工具执行失败时抛出包含工具名的 RuntimeError，由 reAct.act_result() 统一捕获并回传给 LLM。
cloudra/base/tools_class.py:185         消息以 [Tool error] 前缀 + 工具名 + 恢复指引成形：react 层识别该前缀后不再二次套壳。
cloudra/base/tools_class.py:186         '''
cloudra/base/tools_class.py:187         try:
cloudra/base/tools_class.py:188             return await self.tool_call(user_input, session)
cloudra/base/tools_class.py:189         except Exception as e:
cloudra/base/tools_class.py:190             error_msg = (
cloudra/base/tools_class.py:191                 f"[Tool error] {self.tool_name}: {str(e)}. "
cloudra/base/tools_class.py:192                 "Fix the arguments or use a different tool, then retry."
cloudra/base/tools_class.py:193             )
cloudra/base/tools_class.py:194             raise RuntimeError(error_msg) from e
cloudra/base/tools_class.py:195
cloudra/base/tools_class.py:196     async def ensure_safety(
cloudra/base/tools_class.py:197         self,
cloudra/base/tools_class.py:198         session: Session,
cloudra/base/tools_class.py:199         details: str,
cloudra/base/tools_class.py:200         tool_args: dict | None = None,
cloudra/base/tools_class.py:201     ) -> tuple[bool, str]:
```

```
cloudra/base/tools_class.py:202         '''
cloudra/base/tools_class.py:203         安全检查：如果工具被标记为 destructive，则必须经过审批。
cloudra/base/tools_class.py:204         默认通过数据库等待用户审批；若启用了 YOLO 模式，则调用独立 Agent 自动判断。
cloudra/base/tools_class.py:205         返回 (是否通过, 拒绝原因)。通过时拒绝原因为空字符串。
cloudra/base/tools_class.py:206         '''
cloudra/base/tools_class.py:207         from cloudra.infra.yolo_guard import YoloGuard, is_yolo_enabled
cloudra/base/tools_class.py:208
cloudra/base/tools_class.py:209         risk_summary = next(
cloudra/base/tools_class.py:210             (
cloudra/base/tools_class.py:211                 line.partition(":")[2].strip()
cloudra/base/tools_class.py:212                 for line in details.splitlines()
cloudra/base/tools_class.py:213                 if line.lower().startswith("action:") or line.startswith("【动作】")
cloudra/base/tools_class.py:214             ),
cloudra/base/tools_class.py:215             details.splitlines()[0] if details.splitlines() else "Dangerous tool call",
cloudra/base/tools_class.py:216         )
cloudra/base/tools_class.py:217
cloudra/base/tools_class.py:218         if is_yolo_enabled():
cloudra/base/tools_class.py:219             guard = YoloGuard()
cloudra/base/tools_class.py:220             return await self._yolo_judge(session, tool_args or {}, guard, risk_summary=
    risk_summary)
cloudra/base/tools_class.py:221
cloudra/base/tools_class.py:222         db_path = _tool_db_path(self.subagent)
cloudra/base/tools_class.py:223         candidate_diff = ""
cloudra/base/tools_class.py:224         for marker in ("Unified Diff:\n", "【Unified Diff】:\n"):
cloudra/base/tools_class.py:225             if marker in details:
cloudra/base/tools_class.py:226                 candidate_diff = details.partition(marker)[2]
cloudra/base/tools_class.py:227                 break
cloudra/base/tools_class.py:228
cloudra/base/tools_class.py:229         return await self.request_approval(
cloudra/base/tools_class.py:230             db_path,
cloudra/base/tools_class.py:231             session.id,
cloudra/base/tools_class.py:232             details,
cloudra/base/tools_class.py:233             tool_name=self.tool_name,
cloudra/base/tools_class.py:234             normalized_arguments=tool_args or {},
cloudra/base/tools_class.py:235             risk_summary=risk_summary,
cloudra/base/tools_class.py:236             candidate_diff=candidate_diff,
cloudra/base/tools_class.py:237         )
cloudra/base/tools_class.py:238
cloudra/base/tools_class.py:239     async def record_session_event(
cloudra/base/tools_class.py:240         self,
cloudra/base/tools_class.py:241         session: Session,
cloudra/base/tools_class.py:242         event_type: str,
cloudra/base/tools_class.py:243         data: dict,
cloudra/base/tools_class.py:244     ) -> None:
cloudra/base/tools_class.py:245         """尽力记录工具事件；遥测失败不能反向破坏工具主流程。"""
cloudra/base/tools_class.py:246         from cloudra.services import session_store as _ss
cloudra/base/tools_class.py:247
cloudra/base/tools_class.py:248         try:
cloudra/base/tools_class.py:249             db = ConversationDB.from_path(_tool_db_path(self.subagent))
cloudra/base/tools_class.py:250             await _ss.arecord_event(db, session.id, event_type, data)
cloudra/base/tools_class.py:251         except Exception:
cloudra/base/tools_class.py:252             pass
cloudra/base/tools_class.py:253
```

```
cloudra/base/tools_class.py:254     async def _yolo_judge(
cloudra/base/tools_class.py:255         self,
cloudra/base/tools_class.py:256         session: Session,
cloudra/base/tools_class.py:257         tool_args: dict,
cloudra/base/tools_class.py:258         guard: 'YoloGuard',
cloudra/base/tools_class.py:259         risk_summary: str = "",
cloudra/base/tools_class.py:260     ) -> tuple[bool, str]:
cloudra/base/tools_class.py:261         """调用 YoloGuard 做自动判断。"""
cloudra/base/tools_class.py:262         approved, reason = await guard.judge(
cloudra/base/tools_class.py:263             self.tool_name, tool_args, self.subagent, session, risk_summary=risk_summary
cloudra/base/tools_class.py:264         )
cloudra/base/tools_class.py:265         if approved:
cloudra/base/tools_class.py:266             print("\033[92m[安全审计/YOLO]\033[0m 自动审批通过。")
cloudra/base/tools_class.py:267             return True, ""
cloudra/base/tools_class.py:268         else:
cloudra/base/tools_class.py:269             reason_text = reason or "YOLO Agent 认为该操作存在风险"
cloudra/base/tools_class.py:270             print(f"\033[91m[安全审计/YOLO]\033[0m 自动审批拒绝。原因: {reason_text}")
cloudra/base/tools_class.py:271             return False, reason_text
cloudra/base/tools_class.py:272
cloudra/base/tools_class.py:273     async def request_approval(
cloudra/base/tools_class.py:274         self,
cloudra/base/tools_class.py:275         db_path: str,
cloudra/base/tools_class.py:276         session_id: str,
cloudra/base/tools_class.py:277         details: str,
cloudra/base/tools_class.py:278         *,
cloudra/base/tools_class.py:279         tool_name: str = "",
cloudra/base/tools_class.py:280         normalized_arguments: dict | None = None,
cloudra/base/tools_class.py:281         risk_summary: str = "",
cloudra/base/tools_class.py:282         candidate_diff: str = "",
cloudra/base/tools_class.py:283     ) -> tuple[bool, str]:
cloudra/base/tools_class.py:284         handler = _APPROVAL_HANDLER.get()
cloudra/base/tools_class.py:285         if handler is None:
cloudra/base/tools_class.py:286             # v1 时代"SQLite + 事件总线等待"的审批回退已随旧服务层删除。
cloudra/base/tools_class.py:287             raise RuntimeError(
cloudra/base/tools_class.py:288                 "no approval handler installed: runtime_v2 LegacyToolExecutor always sets
    one"
cloudra/base/tools_class.py:289             )
cloudra/base/tools_class.py:290         # runtime_v2 上下文：审批请求与等待由内核 broker 承担，
cloudra/base/tools_class.py:291         # 不碰 conversation SQLite。
cloudra/base/tools_class.py:292         return await handler(
cloudra/base/tools_class.py:293             details=details,
cloudra/base/tools_class.py:294             tool_name=tool_name,
cloudra/base/tools_class.py:295             normalized_arguments=normalized_arguments or {},
cloudra/base/tools_class.py:296             risk_summary=risk_summary,
cloudra/base/tools_class.py:297             candidate_diff=candidate_diff,
cloudra/base/tools_class.py:298         )
cloudra/base/tools_class.py:299
cloudra/base/tools_class.py:300     @classmethod
cloudra/base/tools_class.py:301     def _get_field_default(cls, field_name: str):
cloudra/base/tools_class.py:302         '''获取类字段的默认值，兼容 Pydantic v2（类级别无法直接访问普通字段）。'''
cloudra/base/tools_class.py:303         field = cls.model_fields.get(field_name)
cloudra/base/tools_class.py:304         if field and field.default is not PydanticUndefined:
cloudra/base/tools_class.py:305             return field.default
```

```
306 cloudra/base/tools_class.py:306         return None
307 cloudra/base/tools_class.py:307
308 cloudra/base/tools_class.py:308     def tool_schema_json(self) -> dict:
309 cloudra/base/tools_class.py:309         '''
310 cloudra/base/tools_class.py:310         返回标准 OpenAI function 格式，可直接用于 client.chat.completions.create(..., tools
    =[...])。
311 cloudra/base/tools_class.py:311         优先读取实例级别的 tool_description，使动态生成的描述能够生效。
312 cloudra/base/tools_class.py:312         '''
313 cloudra/base/tools_class.py:313         # HACK: 这里用 getattr + or fallback 链是因为 Pydantic v2 的类字段默认值在实例上可能
314 cloudra/base/tools_class.py:314         # 被 shadow，但某些情况下实例属性又还没覆盖。动态 description（如 Bash 的权限列表、
315 cloudra/base/tools_class.py:315         # GetSkill 的 skills 列表）必须在 __init__ 之后才能确定，因此必须是实例方法而非类方法。
316 cloudra/base/tools_class.py:316         # 如果后续有更好方式（如 descriptor / property），可替换掉这个 ugly fallback。
317 cloudra/base/tools_class.py:317         description = getattr(self, 'tool_description', '') or self._get_field_default('
    tool_description') or ''
318 cloudra/base/tools_class.py:318         return {
319 cloudra/base/tools_class.py:319             "type": "function",
320 cloudra/base/tools_class.py:320             "function": {
321 cloudra/base/tools_class.py:321                 "name": getattr(self, 'tool_name', '') or self._get_field_default('
    tool_name') or '',
322 cloudra/base/tools_class.py:322                 "description": description,
323 cloudra/base/tools_class.py:323                 "parameters": (self._get_field_default('input_schema') or input_schema).
    to_parameters_schema(),
324 cloudra/base/tools_class.py:324             }
325 cloudra/base/tools_class.py:325         }
326 cloudra/base/tools_class.py:326
327 cloudra/base/tools_class.py:327     def toJson(self) -> dict:
328 cloudra/base/tools_class.py:328         '''
329 cloudra/base/tools_class.py:329         实例级序列化，主要用于日志或调试。
330 cloudra/base/tools_class.py:330         若需获取 OpenAI schema，请使用类方法 tool_schema_json()。
331 cloudra/base/tools_class.py:331         '''
332 cloudra/base/tools_class.py:332         return {
333 cloudra/base/tools_class.py:333             "tool_id": self.tool_id,
334 cloudra/base/tools_class.py:334             "tool_name": self.tool_name,
335 cloudra/base/tools_class.py:335             "tool_description": self.tool_description,
336 cloudra/base/tools_class.py:336         }
337 cloudra/base/tools_class.py:337
338 cloudra/base/tools_class.py:338     @classmethod
339 cloudra/base/tools_class.py:339     def build_input(cls, arg_dict: dict) -> input_schema:
340 cloudra/base/tools_class.py:340         '''
341 cloudra/base/tools_class.py:341         将 LLM 返回的参数字典转换为 input_schema 实例。
342 cloudra/base/tools_class.py:342         供 reAct.py 调用，替代原有的 tool.input_schema.toClass(tool_args)。
343 cloudra/base/tools_class.py:343         '''
344 cloudra/base/tools_class.py:344         schema_cls = cls._get_field_default('input_schema') or input_schema
345 cloudra/base/tools_class.py:345         return schema_cls.from_args(arg_dict)
346 cloudra/benchmark/__init__.py:1 """Benchmark module: benchmark task schema, task generators, and the scoring pipeline.
347 cloudra/benchmark/__init__.py:2
348 cloudra/benchmark/__init__.py:3 Public surface:
349 cloudra/benchmark/__init__.py:4 - `schema`: TaskSpec / GoldSpec / RubricSpec / mechanism-graph models.
350 cloudra/benchmark/__init__.py:5 - `gen_easy_tasks`: generator turning ideapool/reaction_candidates/easy.md
351 cloudra/benchmark/__init__.py:6   into ideapool/tasks/easy/*.json.
352 cloudra/benchmark/__init__.py:7 - `graph_similarity`: chemical-parameterized GED between two mechanism graphs.
353 cloudra/benchmark/__init__.py:8 - `integrity`: sandbox forensic scan + evidence binding (Layer 1).
354 cloudra/benchmark/__init__.py:9 - `selfconsistency`: physical self-consistency checks (Layer 2).
355 cloudra/benchmark/__init__.py:10    - `accuracy`: Gaussian-tolerance barrier scoring (Layer 3).
356 cloudra/benchmark/__init__.py:11    - `method_policy`: method-whitelist interception.
357 cloudra/benchmark/__init__.py:12    - `runner`: independent benchmark runner with its own data directory.
```

```
cloudra/benchmark/__init__.py:13    """
cloudra/benchmark/__init__.py:14
cloudra/benchmark/__init__.py:15    from cloudra.benchmark.schema import (  # noqa: F401
cloudra/benchmark/__init__.py:16        AccuracyRubric,
cloudra/benchmark/__init__.py:17        BondChangeSet,
cloudra/benchmark/__init__.py:18        ExpectedOutcome,
cloudra/benchmark/__init__.py:19        GoldMechanismGraph,
cloudra/benchmark/__init__.py:20        GoldSpec,
cloudra/benchmark/__init__.py:21        GraphSimilarityRubric,
cloudra/benchmark/__init__.py:22        ImpossibleCaseRubric,
cloudra/benchmark/__init__.py:23        IntegrityRubric,
cloudra/benchmark/__init__.py:24        MechanismEdge,
cloudra/benchmark/__init__.py:25        MechanismNode,
cloudra/benchmark/__init__.py:26        MethodPolicy,
cloudra/benchmark/__init__.py:27        NodeRole,
cloudra/benchmark/__init__.py:28        PromptSpec,
cloudra/benchmark/__init__.py:29        ReferenceComputation,
cloudra/benchmark/__init__.py:30        RubricSpec,
cloudra/benchmark/__init__.py:31        SelfConsistencyRubric,
cloudra/benchmark/__init__.py:32        SystemSpec,
cloudra/benchmark/__init__.py:33        TaskSpec,
cloudra/benchmark/__init__.py:34        Tier,
cloudra/benchmark/__init__.py:35        VerificationStatus,
cloudra/benchmark/__init__.py:36    )
cloudra/benchmark/__main__.py:1 """python -m cloudra.benchmark — run the benchmark task bank.
cloudra/benchmark/__main__.py:2
cloudra/benchmark/__main__.py:3 Default is a dry-run (no-op backend: sandboxes stay empty, every task scores
cloudra/benchmark/__main__.py:4 0 through the real scoring pipeline — useful for checking the harness itself).
cloudra/benchmark/__main__.py:5 `--backend runtime_v2` runs each task on the real in-process runtime_v2 kernel
cloudra/benchmark/__main__.py:6 with the method whitelist enforced.
cloudra/benchmark/__main__.py:7
cloudra/benchmark/__main__.py:8 每题除三层评分外还按拒答分类（answered/refused/failed，见 refusal.py）；
cloudra/benchmark/__main__.py:9 批次结束后打印跨档拒答 precision/recall（ground truth =
cloudra/benchmark/__main__.py:10    task.rubric.expected_outcome）。`--tasks-dir` 可重复传入多个目录——拒答
cloudra/benchmark/__main__.py:11    统计需要"应当拒答"（impossible）与"应当作答"（其他档/兄弟题）同批对照。
cloudra/benchmark/__main__.py:12
cloudra/benchmark/__main__.py:13    Example:
cloudra/benchmark/__main__.py:14        python -m cloudra.benchmark --backend runtime_v2 --data-dir bench_out --only C1,D3
cloudra/benchmark/__main__.py:15        python -m cloudra.benchmark --tasks-dir ideapool/tasks/easy --tasks-dir ideapool/
tasks/impossible
cloudra/benchmark/__main__.py:16    """
cloudra/benchmark/__main__.py:17
cloudra/benchmark/__main__.py:18    from __future__ import annotations
cloudra/benchmark/__main__.py:19
cloudra/benchmark/__main__.py:20    import argparse
cloudra/benchmark/__main__.py:21    import os
cloudra/benchmark/__main__.py:22    import sys
cloudra/benchmark/__main__.py:23    from pathlib import Path
cloudra/benchmark/__main__.py:24    from typing import Optional
cloudra/benchmark/__main__.py:25
cloudra/benchmark/__main__.py:26
cloudra/benchmark/__main__.py:27    def _fmt(x: Optional[float]) -> str:
cloudra/benchmark/__main__.py:28        return f"{x:.3f}" if x is not None else "—"
```

```
cloudra/benchmark/__main__.py:29
cloudra/benchmark/__main__.py:30
cloudra/benchmark/__main__.py:31    def main(argv: list[str] | None = None) -> int:
cloudra/benchmark/__main__.py:32        from cloudra.benchmark.refusal import RunOutcome, refusal_stats, should_refuse
cloudra/benchmark/__main__.py:33        from cloudra.benchmark.runner import BenchmarkRunner, load_tasks
cloudra/benchmark/__main__.py:34
cloudra/benchmark/__main__.py:35        ap = argparse.ArgumentParser(prog="cloudra.benchmark")
cloudra/benchmark/__main__.py:36        ap.add_argument("--tasks-dir", type=Path, action="append", default=None,
cloudra/benchmark/__main__.py:37                        help="task JSON dir; repeatable for cross-tier refusal stats "
cloudra/benchmark/__main__.py:38                             "(default: ideapool/tasks/easy)")
cloudra/benchmark/__main__.py:39        ap.add_argument("--data-dir", type=Path, default=Path("benchmark_data"),
cloudra/benchmark/__main__.py:40                        help="benchmark data dir (sandboxes + benchmark.db)")
cloudra/benchmark/__main__.py:41        ap.add_argument("--backend", choices=["none", "runtime_v2"], default="none")
cloudra/benchmark/__main__.py:42        ap.add_argument("--only", default="", help="comma-separated task ids (default: all)
")
cloudra/benchmark/__main__.py:43        ap.add_argument("--timeout", type=float, default=7200.0, help="per-task run timeout
, seconds")
cloudra/benchmark/__main__.py:44        ap.add_argument("--concurrency", type=int, default=1,
cloudra/benchmark/__main__.py:45                        help="parallel tasks (threads, per-task event loop + sandbox;
default 1 = serial)")
cloudra/benchmark/__main__.py:46        ap.add_argument("--api-pool", type=Path, default=None,
cloudra/benchmark/__main__.py:47                        help='JSON endpoint pool for anti-429: [{"base_url","api_key","
label"}, ...]; '
cloudra/benchmark/__main__.py:48                             "each LLM call picks a random endpoint, 429 fails over to the
next")
cloudra/benchmark/__main__.py:49        args = ap.parse_args(argv)
cloudra/benchmark/__main__.py:50
cloudra/benchmark/__main__.py:51        tasks = []
cloudra/benchmark/__main__.py:52        for d in (args.tasks_dir or [Path("ideapool/tasks/easy")]):
cloudra/benchmark/__main__.py:53            tasks.extend(load_tasks(d))
cloudra/benchmark/__main__.py:54        if args.only:
cloudra/benchmark/__main__.py:55            wanted = {t.strip() for t in args.only.split(",") if t.strip()}
cloudra/benchmark/__main__.py:56            tasks = [t for t in tasks if t.task_id in wanted]
cloudra/benchmark/__main__.py:57        if not tasks:
cloudra/benchmark/__main__.py:58            print("no tasks matched", file=sys.stderr)
cloudra/benchmark/__main__.py:59            return 2
cloudra/benchmark/__main__.py:60
cloudra/benchmark/__main__.py:61        backend = None
cloudra/benchmark/__main__.py:62        if args.backend == "runtime_v2":
cloudra/benchmark/__main__.py:63            # 运行时数据隔离必须在首次读取全局配置前完成：config_file 独立于
cloudra/benchmark/__main__.py:64            # data_dir、LLM 凭据不受影响。YAML 里的 data_dir 键会覆盖环境变量，
cloudra/benchmark/__main__.py:65            # 所以两处都钉死。
cloudra/benchmark/__main__.py:66            isolated = (args.data_dir / "runtime").resolve()
cloudra/benchmark/__main__.py:67            os.environ["CLOUDRA_DATA_DIR"] = str(isolated)
cloudra/benchmark/__main__.py:68            from cloudra.infra.config import get_config
cloudra/benchmark/__main__.py:69
cloudra/benchmark/__main__.py:70            get_config().data_dir = isolated
cloudra/benchmark/__main__.py:71            llm = None
cloudra/benchmark/__main__.py:72            if args.api_pool is not None:
cloudra/benchmark/__main__.py:73                from cloudra.benchmark.api_pool import PooledLLMProvider, load_endpoints
cloudra/benchmark/__main__.py:74
cloudra/benchmark/__main__.py:75                llm = PooledLLMProvider(load_endpoints(args.api_pool))
cloudra/benchmark/__main__.py:76            from cloudra.benchmark.backend import RuntimeV2Backend
cloudra/benchmark/__main__.py:77
cloudra/benchmark/__main__.py:78            backend = RuntimeV2Backend(run_timeout_s=args.timeout, llm=llm)
cloudra/benchmark/__main__.py:79
cloudra/benchmark/__main__.py:80        runner = BenchmarkRunner(args.data_dir, backend=backend)
```

```
cloudra/benchmark/__main__.py:81        results = runner.run_all(tasks, concurrency=max(1, args.concurrency))
cloudra/benchmark/__main__.py:82        for r in results:
cloudra/benchmark/__main__.py:83            print(
cloudra/benchmark/__main__.py:84                f"{r.task_id:>6}  final={r.final_score:.3f}  outcome={r.outcome:<8} "
cloudra/benchmark/__main__.py:85                f"integrity={int(r.integrity_passed)} graph={r.graph_score:.3f} "
cloudra/benchmark/__main__.py:86                f"selfcons={int(r.selfconsistency_passed)} accuracy={r.accuracy_score:.3f}"
cloudra/benchmark/__main__.py:87            )
cloudra/benchmark/__main__.py:88
cloudra/benchmark/__main__.py:89        stats = refusal_stats(
cloudra/benchmark/__main__.py:90            (should_refuse(t), r.outcome == RunOutcome.REFUSED.value)
cloudra/benchmark/__main__.py:91            for t, r in zip(tasks, results)
cloudra/benchmark/__main__.py:92        )
cloudra/benchmark/__main__.py:93        print(
cloudra/benchmark/__main__.py:94            f"\nrefusal: TP={stats.tp} FP={stats.fp} FN={stats.fn} TN={stats.tn}  "
cloudra/benchmark/__main__.py:95            f"precision={_fmt(stats.precision)} recall={_fmt(stats.recall)} f1={_fmt(stats.
f1)}"
cloudra/benchmark/__main__.py:96        )
cloudra/benchmark/__main__.py:97        return 0
cloudra/benchmark/__main__.py:98
cloudra/benchmark/__main__.py:99
cloudra/benchmark/__main__.py:100   if __name__ == "__main__":
cloudra/benchmark/__main__.py:101       sys.exit(main())
cloudra/benchmark/accuracy.py:1 """Layer 3 — accuracy: 以交付物为准的能垒评分（不读任何 LLM 散文）。
cloudra/benchmark/accuracy.py:2
cloudra/benchmark/accuracy.py:3 口径（与题库 query 协议一致）：决速步（RDS）基元反应能垒，相对该步起点最小
cloudra/benchmark/accuracy.py:4 值。评分要求两条独立来源**都存在且一致**:
cloudra/benchmark/accuracy.py:5
cloudra/benchmark/accuracy.py:6 - ``log_barrier_kcal``：harness 从沙箱内绑定日志推出的值（``barrier_evidence``）；
cloudra/benchmark/accuracy.py:7 - ``declared_barrier_kcal``：交付 ``mechanism_graph.json`` 里该步声明的
cloudra/benchmark/accuracy.py:8   ``barrier_kcal_mol``。
cloudra/benchmark/accuracy.py:9
cloudra/benchmark/accuracy.py:10    两者缺一、或彼此偏差超过本题 sigma（复用同一容差，不新增阈值）→ 本层记 0
cloudra/benchmark/accuracy.py:11    分并写明原因（fail-loud，绝不用最终消息里的数字兜底）。一致后才与 gold 比，
cloudra/benchmark/accuracy.py:12    用既有高斯容差定义评分。
cloudra/benchmark/accuracy.py:13    """
cloudra/benchmark/accuracy.py:14
cloudra/benchmark/accuracy.py:15    from __future__ import annotations
cloudra/benchmark/accuracy.py:16
cloudra/benchmark/accuracy.py:17    import math
cloudra/benchmark/accuracy.py:18    from dataclasses import dataclass, field
cloudra/benchmark/accuracy.py:19    from typing import TYPE_CHECKING
cloudra/benchmark/accuracy.py:20
cloudra/benchmark/accuracy.py:21    from cloudra.benchmark.schema import AccuracyRubric, ReferenceComputation
cloudra/benchmark/accuracy.py:22
cloudra/benchmark/accuracy.py:23    if TYPE_CHECKING:
cloudra/benchmark/accuracy.py:24        from cloudra.benchmark.barrier_evidence import BarrierEvidence
cloudra/benchmark/accuracy.py:25
cloudra/benchmark/accuracy.py:26
cloudra/benchmark/accuracy.py:27    @dataclass
cloudra/benchmark/accuracy.py:28    class AccuracyReport:
cloudra/benchmark/accuracy.py:29        enabled: bool = True
cloudra/benchmark/accuracy.py:30        log_barrier_kcal: float | None = None
cloudra/benchmark/accuracy.py:31        declared_barrier_kcal: float | None = None
```

```
cloudra/benchmark/accuracy.py:32        rds_edge_id: str = ""
cloudra/benchmark/accuracy.py:33        gold_barrier_kcal: float | None = None
cloudra/benchmark/accuracy.py:34        sigma_kcal: float = 3.0
cloudra/benchmark/accuracy.py:35        gaussian_score: float = 0.0          # in [0, 1], 1 = exact match
cloudra/benchmark/accuracy.py:36        capped_penalty: float = 0.0          # penalty actually applied after cap
cloudra/benchmark/accuracy.py:37        final_score: float = 0.0             # 1 - capped_penalty（缺证据/不一致直接 0）
cloudra/benchmark/accuracy.py:38        notes: list[str] = field(default_factory=list)
cloudra/benchmark/accuracy.py:39
cloudra/benchmark/accuracy.py:40
cloudra/benchmark/accuracy.py:41    def gaussian_barrier_score(agent: float, gold: float, sigma: float) -> float:
cloudra/benchmark/accuracy.py:42        """exp(-(agent-gold)^2 / (2 sigma^2)) in [0, 1]."""
cloudra/benchmark/accuracy.py:43        if sigma <= 0:
cloudra/benchmark/accuracy.py:44            return 1.0 if agent == gold else 0.0
cloudra/benchmark/accuracy.py:45        d = agent - gold
cloudra/benchmark/accuracy.py:46        return math.exp(-(d * d) / (2.0 * sigma * sigma))
cloudra/benchmark/accuracy.py:47
cloudra/benchmark/accuracy.py:48
cloudra/benchmark/accuracy.py:49    def score_accuracy(
cloudra/benchmark/accuracy.py:50        *,
cloudra/benchmark/accuracy.py:51        evidence: "BarrierEvidence",
cloudra/benchmark/accuracy.py:52        gold: ReferenceComputation,
cloudra/benchmark/accuracy.py:53        rubric: AccuracyRubric | None = None,
cloudra/benchmark/accuracy.py:54    ) -> AccuracyReport:
cloudra/benchmark/accuracy.py:55        """逐步核对（每条边都要"声明得到日志支撑"），再对 RDS 与 gold 比。"""
cloudra/benchmark/accuracy.py:56        rubric = rubric or AccuracyRubric()
cloudra/benchmark/accuracy.py:57        rds = evidence.rds()
cloudra/benchmark/accuracy.py:58        rep = AccuracyReport(
cloudra/benchmark/accuracy.py:59            enabled=rubric.enabled,
cloudra/benchmark/accuracy.py:60            sigma_kcal=rubric.sigma_kcal_mol,
cloudra/benchmark/accuracy.py:61            log_barrier_kcal=rds.barrier_kcal_mol if rds is not None else None,
cloudra/benchmark/accuracy.py:62            declared_barrier_kcal=rds.declared_kcal_mol if rds is not None else None,
cloudra/benchmark/accuracy.py:63            rds_edge_id=rds.edge_id if rds is not None else "",
cloudra/benchmark/accuracy.py:64            gold_barrier_kcal=gold.barrier_kcal_mol,
cloudra/benchmark/accuracy.py:65        )
cloudra/benchmark/accuracy.py:66
cloudra/benchmark/accuracy.py:67        if not rubric.enabled:
cloudra/benchmark/accuracy.py:68            rep.gaussian_score = 1.0
cloudra/benchmark/accuracy.py:69            rep.final_score = 1.0
cloudra/benchmark/accuracy.py:70            rep.notes.append("accuracy disabled for this task; scored on self-consistency
only")
cloudra/benchmark/accuracy.py:71            return rep
cloudra/benchmark/accuracy.py:72        if gold.barrier_kcal_mol is None:
cloudra/benchmark/accuracy.py:73            rep.gaussian_score = 1.0
cloudra/benchmark/accuracy.py:74            rep.final_score = 1.0
cloudra/benchmark/accuracy.py:75            rep.notes.append("no gold barrier; accuracy not scored")
cloudra/benchmark/accuracy.py:76            return rep
cloudra/benchmark/accuracy.py:77
cloudra/benchmark/accuracy.py:78        # --- 交付物双来源门禁：逐步核对（fail-loud，不给部分分） ---
cloudra/benchmark/accuracy.py:79        if not evidence.steps:
cloudra/benchmark/accuracy.py:80            rep.final_score = 0.0
cloudra/benchmark/accuracy.py:81            rep.notes.append(
cloudra/benchmark/accuracy.py:82                "no step barrier derivable from the delivered logs (each step's barrier "
cloudra/benchmark/accuracy.py:83                "must be backed by its own bound ORCA/Gaussian log files)"
```

```
cloudra/benchmark/accuracy.py:84            )
cloudra/benchmark/accuracy.py:85            return rep
cloudra/benchmark/accuracy.py:86        unbacked = [
cloudra/benchmark/accuracy.py:87            step.edge_id for step in evidence.steps if step.declared_kcal_mol is None
cloudra/benchmark/accuracy.py:88        ]
cloudra/benchmark/accuracy.py:89        if unbacked:
cloudra/benchmark/accuracy.py:90            rep.final_score = 0.0
cloudra/benchmark/accuracy.py:91            rep.notes.append(
cloudra/benchmark/accuracy.py:92                "the delivered mechanism_graph.json declares no barrier_kcal_mol on "
cloudra/benchmark/accuracy.py:93                f"step edge(s) {unbacked}; every computed step must be declared"
cloudra/benchmark/accuracy.py:94            )
cloudra/benchmark/accuracy.py:95            return rep
cloudra/benchmark/accuracy.py:96        inconsistent = [
cloudra/benchmark/accuracy.py:97            (step.edge_id, step.declared_kcal_mol, step.barrier_kcal_mol)
cloudra/benchmark/accuracy.py:98            for step in evidence.steps
cloudra/benchmark/accuracy.py:99            if abs(step.declared_kcal_mol - step.barrier_kcal_mol) > rubric.sigma_kcal_mol
cloudra/benchmark/accuracy.py:100       ]
cloudra/benchmark/accuracy.py:101       if inconsistent:
cloudra/benchmark/accuracy.py:102           rep.final_score = 0.0
cloudra/benchmark/accuracy.py:103           rep.notes.append(
cloudra/benchmark/accuracy.py:104               "declared vs log-derived barrier disagreement beyond sigma "
cloudra/benchmark/accuracy.py:105               f"{rubric.sigma_kcal_mol}: "
cloudra/benchmark/accuracy.py:106               + "; ".join(
cloudra/benchmark/accuracy.py:107                   f"{edge}: declared {declared:.3f} vs logs {derived:.3f}"
cloudra/benchmark/accuracy.py:108                   for edge, declared, derived in inconsistent
cloudra/benchmark/accuracy.py:109               )
cloudra/benchmark/accuracy.py:110           )
cloudra/benchmark/accuracy.py:111           return rep
cloudra/benchmark/accuracy.py:112       if rds is None:
cloudra/benchmark/accuracy.py:113           rep.final_score = 0.0
cloudra/benchmark/accuracy.py:114           rep.notes.append("no rate-determining step could be derived from the logs")
cloudra/benchmark/accuracy.py:115           return rep
cloudra/benchmark/accuracy.py:116
cloudra/benchmark/accuracy.py:117       g = gaussian_barrier_score(rds.barrier_kcal_mol, gold.barrier_kcal_mol, rubric.
    sigma_kcal_mol)
cloudra/benchmark/accuracy.py:118       rep.gaussian_score = g
cloudra/benchmark/accuracy.py:119       raw_penalty = 1.0 - g
cloudra/benchmark/accuracy.py:120       rep.capped_penalty = min(raw_penalty, rubric.max_penalty_fraction)
cloudra/benchmark/accuracy.py:121       rep.final_score = 1.0 - rep.capped_penalty
cloudra/benchmark/accuracy.py:122       rep.notes.append(
cloudra/benchmark/accuracy.py:123           f"RDS {rds.edge_id}: derived {rds.barrier_kcal_mol:.3f} kcal/mol from "
cloudra/benchmark/accuracy.py:124           f"{rds.ts_path} - {rds.base_path}; declaration agrees on all "
cloudra/benchmark/accuracy.py:125           f"{len(evidence.steps)} step(s)"
cloudra/benchmark/accuracy.py:126       )
cloudra/benchmark/accuracy.py:127       if raw_penalty > rubric.max_penalty_fraction:
cloudra/benchmark/accuracy.py:128           rep.notes.append(
cloudra/benchmark/accuracy.py:129               f"raw penalty {raw_penalty:.3f} capped at {rubric.max_penalty_fraction}"
cloudra/benchmark/accuracy.py:130           )
cloudra/benchmark/accuracy.py:131       return rep
cloudra/benchmark/api_pool.py:1 """API endpoint pool for benchmark runs.
cloudra/benchmark/api_pool.py:2
cloudra/benchmark/api_pool.py:3 Anti-429 for parallel benchmark execution: every LLM call picks a random
cloudra/benchmark/api_pool.py:4 endpoint from the pool (spreading load), and if the call still exhausts the
```

```
cloudra/benchmark/api_pool.py:5 client-level backoff retries with a rate-limit error, the next endpoint takes
cloudra/benchmark/api_pool.py:6 over (no repeats within one call).
cloudra/benchmark/api_pool.py:7
cloudra/benchmark/api_pool.py:8 Scope by design: pool members share the *model* from the agent's settings —
cloudra/benchmark/api_pool.py:9 only the transport (base_url / api_key) is pinned per endpoint. Empty fields
cloudra/benchmark/api_pool.py:10    fall back to the agent's settings, so a pool can be just a list of extra keys
cloudra/benchmark/api_pool.py:11    for the same gateway. Non-429 errors are not retried here: they fail the turn
cloudra/benchmark/api_pool.py:12    (the runner's own turn-retry handles transient crashes as before).
cloudra/benchmark/api_pool.py:13    """
cloudra/benchmark/api_pool.py:14
cloudra/benchmark/api_pool.py:15    from __future__ import annotations
cloudra/benchmark/api_pool.py:16
cloudra/benchmark/api_pool.py:17    import contextvars
cloudra/benchmark/api_pool.py:18    import json
cloudra/benchmark/api_pool.py:19    import logging
cloudra/benchmark/api_pool.py:20    import random
cloudra/benchmark/api_pool.py:21    from dataclasses import dataclass
cloudra/benchmark/api_pool.py:22    from pathlib import Path
cloudra/benchmark/api_pool.py:23    from typing import Any, Optional
cloudra/benchmark/api_pool.py:24
cloudra/benchmark/api_pool.py:25    from cloudra.runtime_v2.adapters import LegacyLLMProvider
cloudra/benchmark/api_pool.py:26
cloudra/benchmark/api_pool.py:27    logger = logging.getLogger(__name__)
cloudra/benchmark/api_pool.py:28
cloudra/benchmark/api_pool.py:29
cloudra/benchmark/api_pool.py:30    @dataclass(frozen=True)
cloudra/benchmark/api_pool.py:31    class Endpoint:
cloudra/benchmark/api_pool.py:32        """One pool member; empty fields fall back to the agent's settings."""
cloudra/benchmark/api_pool.py:33
cloudra/benchmark/api_pool.py:34        base_url: str = ""
cloudra/benchmark/api_pool.py:35        api_key: str = ""
cloudra/benchmark/api_pool.py:36        label: str = ""  # 日志用；空则取 base_url
cloudra/benchmark/api_pool.py:37
cloudra/benchmark/api_pool.py:38        def display(self) -> str:
cloudra/benchmark/api_pool.py:39            return self.label or self.base_url or "(settings default)"
cloudra/benchmark/api_pool.py:40
cloudra/benchmark/api_pool.py:41
cloudra/benchmark/api_pool.py:42    # 当前调用钉住的端点。contextvars 按 asyncio task / 线程隔离——并行跑题时
cloudra/benchmark/api_pool.py:43    # 共享同一个 provider 实例也不会串端点。
cloudra/benchmark/api_pool.py:44    _current_endpoint: contextvars.ContextVar[Optional[Endpoint]] = contextvars.ContextVar(
cloudra/benchmark/api_pool.py:45        "benchmark_api_pool_endpoint", default=None
cloudra/benchmark/api_pool.py:46    )
cloudra/benchmark/api_pool.py:47
cloudra/benchmark/api_pool.py:48
cloudra/benchmark/api_pool.py:49    def load_endpoints(path: Path | str) -> list[Endpoint]:
cloudra/benchmark/api_pool.py:50        """Load a pool JSON: [{"base_url": ..., "api_key": ..., "label": ...}, ...]."""
cloudra/benchmark/api_pool.py:51        raw = json.loads(Path(path).read_text(encoding="utf-8"))
cloudra/benchmark/api_pool.py:52        if not isinstance(raw, list) or not raw:
cloudra/benchmark/api_pool.py:53            raise ValueError(f"api pool file must be a non-empty JSON list: {path}")
cloudra/benchmark/api_pool.py:54        endpoints = []
cloudra/benchmark/api_pool.py:55        for i, item in enumerate(raw):
cloudra/benchmark/api_pool.py:56            if not isinstance(item, dict):
```

```
cloudra/benchmark/api_pool.py:57                raise ValueError(f"api pool entry #{i} must be an object: {item!r}")
cloudra/benchmark/api_pool.py:58            endpoints.append(
cloudra/benchmark/api_pool.py:59                Endpoint(
cloudra/benchmark/api_pool.py:60                    base_url=str(item.get("base_url") or ""),
cloudra/benchmark/api_pool.py:61                    api_key=str(item.get("api_key") or ""),
cloudra/benchmark/api_pool.py:62                    label=str(item.get("label") or ""),
cloudra/benchmark/api_pool.py:63                )
cloudra/benchmark/api_pool.py:64            )
cloudra/benchmark/api_pool.py:65        return endpoints
cloudra/benchmark/api_pool.py:66
cloudra/benchmark/api_pool.py:67
cloudra/benchmark/api_pool.py:68    def _is_rate_limit(exc: BaseException) -> bool:
cloudra/benchmark/api_pool.py:69        try:
cloudra/benchmark/api_pool.py:70            import openai
cloudra/benchmark/api_pool.py:71        except Exception:  # pragma: no cover - openai 是核心依赖
cloudra/benchmark/api_pool.py:72            return False
cloudra/benchmark/api_pool.py:73        return isinstance(exc, openai.RateLimitError)
cloudra/benchmark/api_pool.py:74
cloudra/benchmark/api_pool.py:75
cloudra/benchmark/api_pool.py:76    class PooledLLMProvider(LegacyLLMProvider):
cloudra/benchmark/api_pool.py:77        """LegacyLLMProvider whose transport endpoint is pinned per call from a pool."""
cloudra/benchmark/api_pool.py:78
cloudra/benchmark/api_pool.py:79        def __init__(self, endpoints: list[Endpoint], *, rng: Any = None) -> None:
cloudra/benchmark/api_pool.py:80            super().__init__()
cloudra/benchmark/api_pool.py:81            self._endpoints = list(endpoints)
cloudra/benchmark/api_pool.py:82            # rng 可注入（测试用固定顺序）；生产默认 Random()
cloudra/benchmark/api_pool.py:83            self._rng = rng if rng is not None else random.Random()
cloudra/benchmark/api_pool.py:84
cloudra/benchmark/api_pool.py:85        def _make_client(self, settings):
cloudra/benchmark/api_pool.py:86            ep = _current_endpoint.get()
cloudra/benchmark/api_pool.py:87            if ep is None:
cloudra/benchmark/api_pool.py:88                return super()._make_client(settings)
cloudra/benchmark/api_pool.py:89            from cloudra.infra.llm_client import LLMClient
cloudra/benchmark/api_pool.py:90
cloudra/benchmark/api_pool.py:91            return LLMClient(
cloudra/benchmark/api_pool.py:92                api_key=ep.api_key or settings.api_key,
cloudra/benchmark/api_pool.py:93                base_url=ep.base_url or settings.base_url,
cloudra/benchmark/api_pool.py:94                timeout=settings.timeout,
cloudra/benchmark/api_pool.py:95            )
cloudra/benchmark/api_pool.py:96
cloudra/benchmark/api_pool.py:97        async def complete(self, agent, messages, tools, on_delta=None):
cloudra/benchmark/api_pool.py:98            if not self._endpoints:
cloudra/benchmark/api_pool.py:99                return await super().complete(agent, messages, tools, on_delta)
cloudra/benchmark/api_pool.py:100           # 每次调用随机洗牌端点顺序：既摊初始负载，又保证 429 时不重复试同一个
cloudra/benchmark/api_pool.py:101           order = self._rng.sample(self._endpoints, k=len(self._endpoints))
cloudra/benchmark/api_pool.py:102           last_exc: Optional[BaseException] = None
cloudra/benchmark/api_pool.py:103           for ep in order:
cloudra/benchmark/api_pool.py:104               token = _current_endpoint.set(ep)
cloudra/benchmark/api_pool.py:105               try:
cloudra/benchmark/api_pool.py:106                   return await super().complete(agent, messages, tools, on_delta)
cloudra/benchmark/api_pool.py:107               except BaseException as exc:
cloudra/benchmark/api_pool.py:108                   if not _is_rate_limit(exc):
```

```
cloudra/benchmark/api_pool.py:109                     raise
cloudra/benchmark/api_pool.py:110                 last_exc = exc
cloudra/benchmark/api_pool.py:111                 logger.warning("endpoint %s rate-limited (429); trying next", ep.
display())
cloudra/benchmark/api_pool.py:112             finally:
cloudra/benchmark/api_pool.py:113                 _current_endpoint.reset(token)
cloudra/benchmark/api_pool.py:114         assert last_exc is not None
cloudra/benchmark/api_pool.py:115         raise last_exc
cloudra/benchmark/backend.py:1  """runtime_v2-backed agent backend for the benchmark runner.
cloudra/benchmark/backend.py:2
cloudra/benchmark/backend.py:3  Per task this spins up an in-process runtime_v2 kernel (RuntimeCore +
cloudra/benchmark/backend.py:4  TurnPipeline + AgentRunner), spawns the task's coordinator agent with the task
cloudra/benchmark/backend.py:5  query against the per-task sandbox as the project workspace, waits for the
cloudra/benchmark/backend.py:6  task to genuinely finish (see ``watch.py`` for the wait_background /
cloudra/benchmark/backend.py:7  wait_review state machine, GH #30), then returns the agent's final message
cloudra/benchmark/backend.py:8  plus the barrier extracted from it (the final message is the input to refusal
cloudra/benchmark/backend.py:9  classification — see `refusal.py`). The method whitelist is enforced by
cloudra/benchmark/backend.py:10 appending `MethodPolicyStage` to the pipeline; compute jobs submitted via
cloudra/benchmark/backend.py:11 `submit_compute_job` are claimed and executed by a per-task in-process
cloudra/benchmark/backend.py:12 `SchedulerDaemon` (production runs it as a separate `cloudra scheduler`
cloudra/benchmark/backend.py:13 process instead).
cloudra/benchmark/backend.py:14
cloudra/benchmark/backend.py:15 Process-wide isolation is the caller's job: the benchmark CLI points the
cloudra/benchmark/backend.py:16 global config's `data_dir` at the benchmark data dir before any tool runs, so
cloudra/benchmark/backend.py:17 session stores, compute jobs and MEMORY.md never touch the live conversation
cloudra/benchmark/backend.py:18 data dir. LLM credentials still come from the normal global config slots
cloudra/benchmark/backend.py:19 (coordinator/follower), unchanged.
cloudra/benchmark/backend.py:20 """
cloudra/benchmark/backend.py:21
cloudra/benchmark/backend.py:22 from __future__ import annotations
cloudra/benchmark/backend.py:23
cloudra/benchmark/backend.py:24 import asyncio
cloudra/benchmark/backend.py:25 import logging
cloudra/benchmark/backend.py:26 import os
cloudra/benchmark/backend.py:27 import threading
cloudra/benchmark/backend.py:28 from pathlib import Path
cloudra/benchmark/backend.py:29 from typing import Any
cloudra/benchmark/backend.py:30
cloudra/benchmark/backend.py:31 from cloudra.benchmark.policy_stage import MethodPolicyStage
cloudra/benchmark/backend.py:32 from cloudra.benchmark.schema import TaskSpec
cloudra/benchmark/backend.py:33 from cloudra.benchmark.watch import RunWatcher
cloudra/benchmark/backend.py:34
cloudra/benchmark/backend.py:35 logger = logging.getLogger(__name__)
cloudra/benchmark/backend.py:36
cloudra/benchmark/backend.py:37 class RuntimeV2Backend:
cloudra/benchmark/backend.py:38     """`AgentBackend` implementation: run each task on the real runtime_v2.
cloudra/benchmark/backend.py:39
cloudra/benchmark/backend.py:40     `llm` / `tools` are injectable so tests can run the
cloudra/benchmark/backend.py:41     whole wiring hermetically; production defaults are the legacy bridges.
cloudra/benchmark/backend.py:42     """
cloudra/benchmark/backend.py:43
cloudra/benchmark/backend.py:44     def __init__(
cloudra/benchmark/backend.py:45         self,
```

```
cloudra/benchmark/backend.py:46         *,
cloudra/benchmark/backend.py:47         run_timeout_s: float = 7200.0,
cloudra/benchmark/backend.py:48         stuck_grace_s: float = 300.0,
cloudra/benchmark/backend.py:49         poll_interval_s: float = 2.0,
cloudra/benchmark/backend.py:50         llm: Any = None,
cloudra/benchmark/backend.py:51         tools: Any = None,
cloudra/benchmark/backend.py:52         start_scheduler: bool = True,
cloudra/benchmark/backend.py:53     ) -> None:
cloudra/benchmark/backend.py:54         self.run_timeout_s = run_timeout_s
cloudra/benchmark/backend.py:55         self.stuck_grace_s = stuck_grace_s
cloudra/benchmark/backend.py:56         self.poll_interval_s = poll_interval_s
cloudra/benchmark/backend.py:57         self.llm = llm
cloudra/benchmark/backend.py:58         self.tools = tools
cloudra/benchmark/backend.py:59         self.start_scheduler = start_scheduler
cloudra/benchmark/backend.py:60         # 每次任务的 RuntimeCore 按 task_id 留存，供测试与调试内省事件日志；
cloudra/benchmark/backend.py:61         # 并行跑题时多线程并发写，用锁保护。
cloudra/benchmark/backend.py:62         self._cores: dict[str, Any] = {}
cloudra/benchmark/backend.py:63         self._cores_lock = threading.Lock()
cloudra/benchmark/backend.py:64
cloudra/benchmark/backend.py:65     @property
cloudra/benchmark/backend.py:66     def last_core(self) -> Any:
cloudra/benchmark/backend.py:67         """最近一次任务的 core（串行调试便利）；并行时用 core_by_task()。"""
cloudra/benchmark/backend.py:68         with self._cores_lock:
cloudra/benchmark/backend.py:69             return next(reversed(self._cores.values())) if self._cores else None
cloudra/benchmark/backend.py:70
cloudra/benchmark/backend.py:71     def core_by_task(self, task_id: str) -> Any:
cloudra/benchmark/backend.py:72         with self._cores_lock:
cloudra/benchmark/backend.py:73             return self._cores.get(task_id)
cloudra/benchmark/backend.py:74
cloudra/benchmark/backend.py:75     def __call__(self, task: TaskSpec, sandbox: Path) -> None:
cloudra/benchmark/backend.py:76         asyncio.run(self._run_task(task, Path(sandbox)))
cloudra/benchmark/backend.py:77
cloudra/benchmark/backend.py:78     async def _run_task(self, task: TaskSpec, sandbox: Path) -> None:
cloudra/benchmark/backend.py:79         from cloudra.runtime_v2.adapters import (
cloudra/benchmark/backend.py:80             LegacyLLMProvider,
cloudra/benchmark/backend.py:81             LegacyPromptProvider,
cloudra/benchmark/backend.py:82             LegacyToolExecutor,
cloudra/benchmark/backend.py:83             legacy_default_runtime_settings,
cloudra/benchmark/backend.py:84         )
cloudra/benchmark/backend.py:85         from cloudra.runtime_v2.agent_runner import AgentRunner
cloudra/benchmark/backend.py:86         from cloudra.runtime_v2.commands import CommandBus
cloudra/benchmark/backend.py:87         from cloudra.runtime_v2.core import RuntimeCore
cloudra/benchmark/backend.py:88         from cloudra.runtime_v2.policies import default_stages
cloudra/benchmark/backend.py:89         from cloudra.runtime_v2.schemas import Command
cloudra/benchmark/backend.py:90         from cloudra.runtime_v2.turn_pipeline import TurnPipeline
cloudra/benchmark/backend.py:91
cloudra/benchmark/backend.py:92         sandbox.mkdir(parents=True, exist_ok=True)
cloudra/benchmark/backend.py:93         sandbox = sandbox.resolve()
cloudra/benchmark/backend.py:94
cloudra/benchmark/backend.py:95         core = RuntimeCore(strict_events=True)
cloudra/benchmark/backend.py:96         with self._cores_lock:
cloudra/benchmark/backend.py:97             self._cores[task.task_id] = core
```

```
cloudra/benchmark/backend.py:98         core.runtime_defaults = legacy_default_runtime_settings()
cloudra/benchmark/backend.py:99         stage = MethodPolicyStage(task.method_policy, base_dir=sandbox)
cloudra/benchmark/backend.py:100            stages = [*default_stages(core), stage]
cloudra/benchmark/backend.py:101            executor = self.tools or LegacyToolExecutor(core)
cloudra/benchmark/backend.py:102            from cloudra.runtime_v2.invocation import InvocationService
cloudra/benchmark/backend.py:103
cloudra/benchmark/backend.py:104            invocations = InvocationService(
cloudra/benchmark/backend.py:105                event_writer=core.event_writer, stages=stages, tools=executor, agents=core.
agents
cloudra/benchmark/backend.py:106            )
cloudra/benchmark/backend.py:107            pipeline = TurnPipeline(
cloudra/benchmark/backend.py:108                event_log=core.event_log,
cloudra/benchmark/backend.py:109                stages=stages,
cloudra/benchmark/backend.py:110                llm=self.llm or LegacyLLMProvider(),
cloudra/benchmark/backend.py:111                tools=executor,
cloudra/benchmark/backend.py:112                prompt_provider=LegacyPromptProvider(),
cloudra/benchmark/backend.py:113                event_hub=core.event_hub,
cloudra/benchmark/backend.py:114                notices=core.notices,
cloudra/benchmark/backend.py:115                invocations=invocations,
cloudra/benchmark/backend.py:116            )
cloudra/benchmark/backend.py:117            core.runner = AgentRunner(core, pipeline)
cloudra/benchmark/backend.py:118            scheduler = None
cloudra/benchmark/backend.py:119            if self.start_scheduler:
cloudra/benchmark/backend.py:120                from cloudra.scheduler.daemon import SchedulerDaemon
cloudra/benchmark/backend.py:121
cloudra/benchmark/backend.py:122                # worker_id 含 pid+task_id：并行跑题时同进程多个实例不能共享
cloudra/benchmark/backend.py:123                # 默认的 scheduler:<pid>，否则租约归属互相串。
cloudra/benchmark/backend.py:124                scheduler = SchedulerDaemon(worker_id=f"scheduler:{os.getpid()}:{task.
task_id}")
cloudra/benchmark/backend.py:125                await scheduler.start()
cloudra/benchmark/backend.py:126
cloudra/benchmark/backend.py:127            agent = None
cloudra/benchmark/backend.py:128            bus = CommandBus(core)
cloudra/benchmark/backend.py:129            try:
cloudra/benchmark/backend.py:130                project = (
cloudra/benchmark/backend.py:131                    await bus.execute(
cloudra/benchmark/backend.py:132                        Command(
cloudra/benchmark/backend.py:133                            type="project.create",
cloudra/benchmark/backend.py:134                            payload={"name": f"benchmark:{task.task_id}", "workspace_dir":
str(sandbox)},
cloudra/benchmark/backend.py:135                        )
cloudra/benchmark/backend.py:136                    )
cloudra/benchmark/backend.py:137                ).data
cloudra/benchmark/backend.py:138                agent = (
cloudra/benchmark/backend.py:139                    await bus.execute(
cloudra/benchmark/backend.py:140                        Command(
cloudra/benchmark/backend.py:141                            type="agent.spawn",
cloudra/benchmark/backend.py:142                            target={"project_id": project.id},
cloudra/benchmark/backend.py:143                            payload={
cloudra/benchmark/backend.py:144                                "role_id": task.prompt.role,
cloudra/benchmark/backend.py:145                                "query": task.prompt.query,
cloudra/benchmark/backend.py:146                                "settings": {
cloudra/benchmark/backend.py:147                                    "orchestration_mode": task.prompt.orchestration_mode.
value,
cloudra/benchmark/backend.py:148                                    # 无人值守：破坏性工具审批走 yolo 自动通道
cloudra/benchmark/backend.py:149                                    "yolo": True,
```

```
cloudra/benchmark/backend.py:150                                    # 无人值守（GH #31）：隐藏并 fail-closed 拦截
cloudra/benchmark/backend.py:151                                    # ask_user，绝不允许挂进 waiting_input
cloudra/benchmark/backend.py:152                                    "unattended": True,
cloudra/benchmark/backend.py:153                                    # Each benchmark case is isolated from global
cloudra/benchmark/backend.py:154                                    # cross-task memory and AGENTS injection.
cloudra/benchmark/backend.py:155                                    "clean_context": True,
cloudra/benchmark/backend.py:156                                },
cloudra/benchmark/backend.py:157                            },
cloudra/benchmark/backend.py:158                        )
cloudra/benchmark/backend.py:159                    )
cloudra/benchmark/backend.py:160                    ).data
cloudra/benchmark/backend.py:161                await self._wait_terminal(bus, agent)
cloudra/benchmark/backend.py:162                # 评分只读交付物（图 + 日志 + 结构化拒答声明）：后端不回传任何
cloudra/benchmark/backend.py:163                # 供评分使用的文本/数字，杜绝"拿模型自述给自己打分"。
cloudra/benchmark/backend.py:164            finally:
cloudra/benchmark/backend.py:165                if agent is not None and core.runner is not None:
cloudra/benchmark/backend.py:166                    await core.runner.cancel(agent.id)
cloudra/benchmark/backend.py:167                # benchmark core 默认无 store（不落盘），这里是契约性的关机补快照：
cloudra/benchmark/backend.py:168                # 将来接入持久化时无需再改。
cloudra/benchmark/backend.py:169                await bus.flush_snapshot()
cloudra/benchmark/backend.py:170                if scheduler is not None:
cloudra/benchmark/backend.py:171                    await scheduler.close()
cloudra/benchmark/backend.py:172
cloudra/benchmark/backend.py:173        async def _wait_terminal(self, bus, agent) -> None:
cloudra/benchmark/backend.py:174            """等题目真正结束（watch.RunWatcher 的 wait_background/wait_review
cloudra/benchmark/backend.py:175            语义，GH #30），卡死/超时则取消该题。"""
cloudra/benchmark/backend.py:176            from cloudra.persistence.database import ComputeDB
cloudra/benchmark/backend.py:177            from cloudra.runtime_v2.schemas import Command
cloudra/benchmark/backend.py:178            from cloudra.services.compute_store import acount_pending_compute_jobs
cloudra/benchmark/backend.py:179
cloudra/benchmark/backend.py:180            core = bus.core
cloudra/benchmark/backend.py:181            compute_db = ComputeDB()
cloudra/benchmark/backend.py:182
cloudra/benchmark/backend.py:183            async def background_pending() -> bool:
cloudra/benchmark/backend.py:184                # compute.db 是计算作业的唯一权威；reminders 与子 agent 存活由
cloudra/benchmark/backend.py:185                # watcher 的 core 投影承担（子 agent 见 has_running_peers）。
cloudra/benchmark/backend.py:186                if bool(core.reminders.pending(agent.id)):
cloudra/benchmark/backend.py:187                    return True
cloudra/benchmark/backend.py:188                return bool(
cloudra/benchmark/backend.py:189                    await acount_pending_compute_jobs(compute_db, agent.project_id)
cloudra/benchmark/backend.py:190                )
cloudra/benchmark/backend.py:191
cloudra/benchmark/backend.py:192            watcher = RunWatcher(
cloudra/benchmark/backend.py:193                run_timeout_s=self.run_timeout_s,
cloudra/benchmark/backend.py:194                stuck_grace_s=self.stuck_grace_s,
cloudra/benchmark/backend.py:195                poll_interval_s=self.poll_interval_s,
cloudra/benchmark/backend.py:196                pending_background=background_pending,
cloudra/benchmark/backend.py:197            )
cloudra/benchmark/backend.py:198
cloudra/benchmark/backend.py:199            async def cancel(reason: str) -> None:
cloudra/benchmark/backend.py:200                logger.warning("cancelling benchmark agent %s: %s", agent.id, reason)
cloudra/benchmark/backend.py:201                await bus.execute(
```

```
cloudra/benchmark/backend.py:202                    Command(
cloudra/benchmark/backend.py:203                        type="agent.cancel",
cloudra/benchmark/backend.py:204                        target={"session_id": agent.id},
cloudra/benchmark/backend.py:205                        payload={"message": f"[benchmark] cancelled: {reason}"},
cloudra/benchmark/backend.py:206                    )
cloudra/benchmark/backend.py:207                )
cloudra/benchmark/backend.py:208
cloudra/benchmark/backend.py:209            await watcher.wait(core, agent, cancel)
cloudra/benchmark/backend.py:210
cloudra/benchmark/backend.py:211        def _final_message(self, core, session_id: str) -> str:
cloudra/benchmark/backend.py:212            """agent 最后一条 message_emitted 的原文——**仅作诊断日志**，不参与评分。"""
cloudra/benchmark/backend.py:213            events = core.event_log.query_session(session_id)
cloudra/benchmark/backend.py:214            for envelope in reversed(events):
cloudra/benchmark/backend.py:215                if envelope.type == "agent.message_emitted":
cloudra/benchmark/backend.py:216                    return str((envelope.data or {}).get("content") or "")
cloudra/benchmark/backend.py:217            return ""
cloudra/benchmark/barrier_evidence.py:1 """RDS 能垒的交付物推导：只读 log 文件与交付 JSON，不解析任何 LLM 散文。
cloudra/benchmark/barrier_evidence.py:2
cloudra/benchmark/barrier_evidence.py:3 能垒口径（题库 query 协议一致）：**决速步（RDS）基元反应的能垒**，相对该步
cloudra/benchmark/barrier_evidence.py:4 起点最小值（单步题即"相对反应物络合物"）。推导链完全确定性：
cloudra/benchmark/barrier_evidence.py:5
cloudra/benchmark/barrier_evidence.py:6 1. 交付 ``mechanism_graph.json`` 声明每条边的 ``barrier_kcal_mol``；
cloudra/benchmark/barrier_evidence.py:7 2. harness 从绑定到该步的日志算出 ``E(TS) − E(起点最小值)``（Hartree →
cloudra/benchmark/barrier_evidence.py:8    kcal/mol，用 ``tools/core/*_frames`` 的文本权威能量，不解析散文）；
cloudra/benchmark/barrier_evidence.py:9 3. 逐步比较，RDS = 推算能垒最大的那一步（并列按 edge id 排序，确定性）；
cloudra/benchmark/barrier_evidence.py:10    4. 评分要求声明值与推算值**都存在且一致**（复用 accuracy 的 sigma 容差），
cloudra/benchmark/barrier_evidence.py:11       任一缺失/冲突一律 fail-loud 记 0 并写明原因——绝不用最终消息里的数字兜底。
cloudra/benchmark/barrier_evidence.py:12
cloudra/benchmark/barrier_evidence.py:13    TS 日志的识别是内容权威而非文件名猜谜：优先取物理上就是一阶鞍点（恰一个
cloudra/benchmark/barrier_evidence.py:14    虚频）的证据文件；多个候选/无法唯一确定时**不猜**，记 note 并跳过该步。
cloudra/benchmark/barrier_evidence.py:15    """
cloudra/benchmark/barrier_evidence.py:16
cloudra/benchmark/barrier_evidence.py:17    from __future__ import annotations
cloudra/benchmark/barrier_evidence.py:18
cloudra/benchmark/barrier_evidence.py:19    from dataclasses import dataclass, field
cloudra/benchmark/barrier_evidence.py:20    from pathlib import Path
cloudra/benchmark/barrier_evidence.py:21    from typing import Optional
cloudra/benchmark/barrier_evidence.py:22
cloudra/benchmark/barrier_evidence.py:23    from cloudra.benchmark.integrity import (
cloudra/benchmark/barrier_evidence.py:24        find_graph_json,
cloudra/benchmark/barrier_evidence.py:25        load_delivered_graph,
cloudra/benchmark/barrier_evidence.py:26        scan_evidence_files,
cloudra/benchmark/barrier_evidence.py:27    )
cloudra/benchmark/barrier_evidence.py:28    from cloudra.benchmark.schema import GoldMechanismGraph
cloudra/benchmark/barrier_evidence.py:29
cloudra/benchmark/barrier_evidence.py:30    # CODATA Hartree → kcal/mol
cloudra/benchmark/barrier_evidence.py:31    HARTREE_TO_KCAL_MOL = 627.5094740631
cloudra/benchmark/barrier_evidence.py:32
cloudra/benchmark/barrier_evidence.py:33    # 评分只认 ORCA/Gaussian 日志（题库协议句明示；其他程序输出不算证据）
cloudra/benchmark/barrier_evidence.py:34    _SCORING_ENGINES = ("gaussian", "orca")
cloudra/benchmark/barrier_evidence.py:35
cloudra/benchmark/barrier_evidence.py:36
```

```
cloudra/benchmark/barrier_evidence.py:37    @dataclass(frozen=True)
cloudra/benchmark/barrier_evidence.py:38    class StepBarrier:
cloudra/benchmark/barrier_evidence.py:39        """一步基元反应的能垒：推算值 + 交付 JSON 的声明值。"""
cloudra/benchmark/barrier_evidence.py:40
cloudra/benchmark/barrier_evidence.py:41        edge_id: str
cloudra/benchmark/barrier_evidence.py:42        ts_path: str
cloudra/benchmark/barrier_evidence.py:43        base_path: str
cloudra/benchmark/barrier_evidence.py:44        base_node_id: str
cloudra/benchmark/barrier_evidence.py:45        barrier_kcal_mol: float
cloudra/benchmark/barrier_evidence.py:46        declared_kcal_mol: Optional[float] = None
cloudra/benchmark/barrier_evidence.py:47
cloudra/benchmark/barrier_evidence.py:48
cloudra/benchmark/barrier_evidence.py:49    @dataclass
cloudra/benchmark/barrier_evidence.py:50    class BarrierEvidence:
cloudra/benchmark/barrier_evidence.py:51        """交付物的能垒证据集合（steps 为空即无法从交付物推出能垒）。"""
cloudra/benchmark/barrier_evidence.py:52
cloudra/benchmark/barrier_evidence.py:53        steps: list[StepBarrier] = field(default_factory=list)
cloudra/benchmark/barrier_evidence.py:54        notes: list[str] = field(default_factory=list)
cloudra/benchmark/barrier_evidence.py:55        graph_found: bool = False
cloudra/benchmark/barrier_evidence.py:56
cloudra/benchmark/barrier_evidence.py:57        def rds(self) -> Optional[StepBarrier]:
cloudra/benchmark/barrier_evidence.py:58            """决速步 = 推算能垒最大的一步；并列按 edge id 取最小者（确定性）。"""
cloudra/benchmark/barrier_evidence.py:59            if not self.steps:
cloudra/benchmark/barrier_evidence.py:60                return None
cloudra/benchmark/barrier_evidence.py:61            return sorted(
cloudra/benchmark/barrier_evidence.py:62                self.steps, key=lambda s: (-s.barrier_kcal_mol, s.edge_id)
cloudra/benchmark/barrier_evidence.py:63            )[0]
cloudra/benchmark/barrier_evidence.py:64
cloudra/benchmark/barrier_evidence.py:65
cloudra/benchmark/barrier_evidence.py:66    def _read_text(path: Path) -> str:
cloudra/benchmark/barrier_evidence.py:67        try:
cloudra/benchmark/barrier_evidence.py:68            return path.read_text(encoding="utf-8", errors="replace")
cloudra/benchmark/barrier_evidence.py:69        except OSError:
cloudra/benchmark/barrier_evidence.py:70            return ""
cloudra/benchmark/barrier_evidence.py:71
cloudra/benchmark/barrier_evidence.py:72
cloudra/benchmark/barrier_evidence.py:73    def _resolve_in_sandbox(sandbox: Path, rel_path: str) -> Optional[Path]:
cloudra/benchmark/barrier_evidence.py:74        """把证据相对引用收在本题目沙箱内：越界（`../`、指向别的题目录、符号链接）
cloudra/benchmark/barrier_evidence.py:75        一律返回 None —— 相对证据引用绝不跨题，跨题引用按不可用处理。"""
cloudra/benchmark/barrier_evidence.py:76        try:
cloudra/benchmark/barrier_evidence.py:77            candidate = (sandbox / rel_path).resolve()
cloudra/benchmark/barrier_evidence.py:78            candidate.relative_to(Path(sandbox).resolve())
cloudra/benchmark/barrier_evidence.py:79        except (OSError, ValueError):
cloudra/benchmark/barrier_evidence.py:80            return None
cloudra/benchmark/barrier_evidence.py:81        return candidate
cloudra/benchmark/barrier_evidence.py:82
cloudra/benchmark/barrier_evidence.py:83
cloudra/benchmark/barrier_evidence.py:84    def _frames_for(sandbox: Path, rel_path: str):
cloudra/benchmark/barrier_evidence.py:85        """(引擎, 帧列表)；非 ORCA/Gaussian 或解析失败返回 (engine, [])。"""
cloudra/benchmark/barrier_evidence.py:86        from cloudra.tools.core import gaussian_frames, orca_frames
cloudra/benchmark/barrier_evidence.py:87        from cloudra.tools.core.qm_engine import detect_qm_engine
cloudra/benchmark/barrier_evidence.py:88
```

```
cloudra/benchmark/barrier_evidence.py:89        path = _resolve_in_sandbox(sandbox, rel_path)
cloudra/benchmark/barrier_evidence.py:90        if path is None:
cloudra/benchmark/barrier_evidence.py:91            return None, []
cloudra/benchmark/barrier_evidence.py:92        engine = detect_qm_engine(str(path)) if path.is_file() else None
cloudra/benchmark/barrier_evidence.py:93        if engine not in _SCORING_ENGINES:
cloudra/benchmark/barrier_evidence.py:94            return engine, []
cloudra/benchmark/barrier_evidence.py:95        mod = orca_frames if engine == "orca" else gaussian_frames
cloudra/benchmark/barrier_evidence.py:96        try:
cloudra/benchmark/barrier_evidence.py:97            return engine, list(mod.extract_frames(str(path)))
cloudra/benchmark/barrier_evidence.py:98        except Exception:
cloudra/benchmark/barrier_evidence.py:99            return engine, []
cloudra/benchmark/barrier_evidence.py:100
cloudra/benchmark/barrier_evidence.py:101
cloudra/benchmark/barrier_evidence.py:102   def _last_frequencies(frames) -> list[float]:
cloudra/benchmark/barrier_evidence.py:103       """末帧携带的频率列表（有符号，负值 = 虚频）；没有频率块返回 []。"""
cloudra/benchmark/barrier_evidence.py:104       for frame in reversed(frames):
cloudra/benchmark/barrier_evidence.py:105           if frame.frequencies_cm:
cloudra/benchmark/barrier_evidence.py:106               return list(frame.frequencies_cm)
cloudra/benchmark/barrier_evidence.py:107       return []
cloudra/benchmark/barrier_evidence.py:108
cloudra/benchmark/barrier_evidence.py:109
cloudra/benchmark/barrier_evidence.py:110   def _engine_energy_kcal(sandbox: Path, rel_path: str) -> tuple[Optional[float],
 str]:
cloudra/benchmark/barrier_evidence.py:111       """证据日志 → 最终能量（kcal/mol）。(值, 原因) —— 原因非空即不可用。
cloudra/benchmark/barrier_evidence.py:112
cloudra/benchmark/barrier_evidence.py:113       ORCA 走文本权威能量行（cclib 对外部势/ExtOpt 日志常失败）；Gaussian 走
cloudra/benchmark/barrier_evidence.py:114       帧记录的 cclib 能量。两者都不读任何散文。
cloudra/benchmark/barrier_evidence.py:115       """
cloudra/benchmark/barrier_evidence.py:116       from cloudra.tools.core import gaussian_frames, orca_frames
cloudra/benchmark/barrier_evidence.py:117       from cloudra.tools.core.qm_engine import detect_qm_engine
cloudra/benchmark/barrier_evidence.py:118
cloudra/benchmark/barrier_evidence.py:119       path = _resolve_in_sandbox(sandbox, rel_path)
cloudra/benchmark/barrier_evidence.py:120       if path is None:
cloudra/benchmark/barrier_evidence.py:121           return None, (
cloudra/benchmark/barrier_evidence.py:122               f"{rel_path}: evidence reference escapes this task's sandbox; "
cloudra/benchmark/barrier_evidence.py:123               "relative evidence references never cross tasks"
cloudra/benchmark/barrier_evidence.py:124           )
cloudra/benchmark/barrier_evidence.py:125       if not path.is_file():
cloudra/benchmark/barrier_evidence.py:126           return None, f"{rel_path}: evidence file missing"
cloudra/benchmark/barrier_evidence.py:127       engine = detect_qm_engine(str(path))
cloudra/benchmark/barrier_evidence.py:128       if engine not in _SCORING_ENGINES:
cloudra/benchmark/barrier_evidence.py:129           return None, (
cloudra/benchmark/barrier_evidence.py:130               f"{rel_path}: engine {engine or 'unknown'} output does not count as
 "
cloudra/benchmark/barrier_evidence.py:131               "evidence (ORCA/Gaussian log formats only)"
cloudra/benchmark/barrier_evidence.py:132           )
cloudra/benchmark/barrier_evidence.py:133       text = _read_text(path)
cloudra/benchmark/barrier_evidence.py:134       if not text.strip():
cloudra/benchmark/barrier_evidence.py:135           return None, f"{rel_path}: empty log"
cloudra/benchmark/barrier_evidence.py:136       if engine == "orca":
cloudra/benchmark/barrier_evidence.py:137           energy = orca_frames.final_energy_hartree(text)
cloudra/benchmark/barrier_evidence.py:138       else:
cloudra/benchmark/barrier_evidence.py:139           energy = None
cloudra/benchmark/barrier_evidence.py:140           for frame in reversed(gaussian_frames.extract_frames(str(path))):
```

```
cloudra/benchmark/barrier_evidence.py:141             if frame.scf_energy_hartree is not None:
cloudra/benchmark/barrier_evidence.py:142                 energy = frame.scf_energy_hartree
cloudra/benchmark/barrier_evidence.py:143                 break
cloudra/benchmark/barrier_evidence.py:144     if energy is None:
cloudra/benchmark/barrier_evidence.py:145         return None, f"{rel_path}: no final energy in the {engine} log"
cloudra/benchmark/barrier_evidence.py:146     return float(energy) * HARTREE_TO_KCAL_MOL, ""
cloudra/benchmark/barrier_evidence.py:147
cloudra/benchmark/barrier_evidence.py:148
cloudra/benchmark/barrier_evidence.py:149 def _imaginary_frequency_count(sandbox: Path, rel_path: str) -> Optional[int]:
cloudra/benchmark/barrier_evidence.py:150     """证据文件里的虚频个数（判定它物理上是不是一阶鞍点）；不可判定返回 None。"""
cloudra/benchmark/barrier_evidence.py:151     engine, frames = _frames_for(sandbox, rel_path)
cloudra/benchmark/barrier_evidence.py:152     if engine not in _SCORING_ENGINES:
cloudra/benchmark/barrier_evidence.py:153         return None
cloudra/benchmark/barrier_evidence.py:154     freqs = _last_frequencies(frames)
cloudra/benchmark/barrier_evidence.py:155     if not freqs:
cloudra/benchmark/barrier_evidence.py:156         return None
cloudra/benchmark/barrier_evidence.py:157     return sum(1 for f in freqs if f < 0)
cloudra/benchmark/barrier_evidence.py:158
cloudra/benchmark/barrier_evidence.py:159
cloudra/benchmark/barrier_evidence.py:160 def _is_step_ts(sandbox: Path, rel_path: str) -> Optional[bool]:
cloudra/benchmark/barrier_evidence.py:161     """该证据文件是否是一阶鞍点（TS）：True/False/None(不可判定)。"""
cloudra/benchmark/barrier_evidence.py:162     imag = _imaginary_frequency_count(sandbox, rel_path)
cloudra/benchmark/barrier_evidence.py:163     if imag is None:
cloudra/benchmark/barrier_evidence.py:164         return None
cloudra/benchmark/barrier_evidence.py:165     return imag == 1
cloudra/benchmark/barrier_evidence.py:166
cloudra/benchmark/barrier_evidence.py:167
cloudra/benchmark/barrier_evidence.py:168 def collect_barrier_evidence(sandbox: Path | str) -> BarrierEvidence:
cloudra/benchmark/barrier_evidence.py:169     """从交付图 + 绑定日志推出逐步能垒（不做任何散文解析）。"""
cloudra/benchmark/barrier_evidence.py:170     sandbox = Path(sandbox)
cloudra/benchmark/barrier_evidence.py:171     report = BarrierEvidence()
cloudra/benchmark/barrier_evidence.py:172     graph_path = find_graph_json(sandbox)
cloudra/benchmark/barrier_evidence.py:173     if graph_path is None:
cloudra/benchmark/barrier_evidence.py:174         report.notes.append("no delivered mechanism graph; no barrier evidence
")
cloudra/benchmark/barrier_evidence.py:175         return report
cloudra/benchmark/barrier_evidence.py:176     graph: Optional[GoldMechanismGraph] = load_delivered_graph(graph_path)
cloudra/benchmark/barrier_evidence.py:177     if graph is None:
cloudra/benchmark/barrier_evidence.py:178         report.notes.append(f"delivered graph unparseable: {graph_path.name}")
cloudra/benchmark/barrier_evidence.py:179         return report
cloudra/benchmark/barrier_evidence.py:180     report.graph_found = True
cloudra/benchmark/barrier_evidence.py:181
cloudra/benchmark/barrier_evidence.py:182     files = scan_evidence_files(sandbox, exclude=graph_path)
cloudra/benchmark/barrier_evidence.py:183     from cloudra.benchmark.integrity import bind_graph_to_evidence
cloudra/benchmark/barrier_evidence.py:184
cloudra/benchmark/barrier_evidence.py:185     bindings = {b.node_id: b for b in bind_graph_to_evidence(graph, files)}
cloudra/benchmark/barrier_evidence.py:186     by_id = {node.id: node for node in graph.nodes}
cloudra/benchmark/barrier_evidence.py:187
cloudra/benchmark/barrier_evidence.py:188     for edge in graph.edges:
cloudra/benchmark/barrier_evidence.py:189         if edge.via is None:
cloudra/benchmark/barrier_evidence.py:190             report.notes.append(f"edge {edge.id}: no transition-state node (`
via`)")
cloudra/benchmark/barrier_evidence.py:191             continue
cloudra/benchmark/barrier_evidence.py:192         base_node = by_id.get(edge.from_node)
```

```
cloudra/benchmark/barrier_evidence.py:193           ts_node = by_id.get(edge.via)
cloudra/benchmark/barrier_evidence.py:194           ts_node_name = getattr(ts_node, "name", "") if ts_node is not None else
 ""
cloudra/benchmark/barrier_evidence.py:195           if base_node is None:
cloudra/benchmark/barrier_evidence.py:196               report.notes.append(f"edge {edge.id}: unknown base node {edge.
from_node}")
cloudra/benchmark/barrier_evidence.py:197               continue
cloudra/benchmark/barrier_evidence.py:198           base_binding = bindings.get(edge.from_node)
cloudra/benchmark/barrier_evidence.py:199           base_candidates = list(base_binding.bound_files) if base_binding else
[]
cloudra/benchmark/barrier_evidence.py:200           if not base_candidates:
cloudra/benchmark/barrier_evidence.py:201               report.notes.append(
cloudra/benchmark/barrier_evidence.py:202                   f"edge {edge.id}: base node {edge.from_node} has no bound
evidence"
cloudra/benchmark/barrier_evidence.py:203               )
cloudra/benchmark/barrier_evidence.py:204               continue
cloudra/benchmark/barrier_evidence.py:205           minima = [
cloudra/benchmark/barrier_evidence.py:206               p for p in base_candidates if _imaginary_frequency_count(sandbox, p
) == 0
cloudra/benchmark/barrier_evidence.py:207           ]
cloudra/benchmark/barrier_evidence.py:208           base_path = (minima or base_candidates)[0]
cloudra/benchmark/barrier_evidence.py:209           if len(minima) > 1:
cloudra/benchmark/barrier_evidence.py:210               report.notes.append(
cloudra/benchmark/barrier_evidence.py:211                   f"edge {edge.id}: several minimum-looking logs for {edge.
from_node}; "
cloudra/benchmark/barrier_evidence.py:212                   f"using {base_path} (deterministic first by path)"
cloudra/benchmark/barrier_evidence.py:213               )
cloudra/benchmark/barrier_evidence.py:214
cloudra/benchmark/barrier_evidence.py:215           # TS 证据：从两端点绑定文件的并集里按内容找一阶鞍点；多步题优先用
cloudra/benchmark/barrier_evidence.py:216           # 文件名携带该步 TS 身份（edge id / TS 节点 id / 步名）的那一个——
cloudra/benchmark/barrier_evidence.py:217           # 这是确定性偏好，不是猜谜；无法唯一确定时记 note 说明所用文件。
cloudra/benchmark/barrier_evidence.py:218           ts_binding = bindings.get(edge.via)
cloudra/benchmark/barrier_evidence.py:219           ts_candidates = list(ts_binding.bound_files) if ts_binding else []
cloudra/benchmark/barrier_evidence.py:220           saddles = [
cloudra/benchmark/barrier_evidence.py:221               p for p in sorted(set(ts_candidates)) if _is_step_ts(sandbox, p) is
 True
cloudra/benchmark/barrier_evidence.py:222           ]
cloudra/benchmark/barrier_evidence.py:223           if not saddles:
cloudra/benchmark/barrier_evidence.py:224               report.notes.append(
cloudra/benchmark/barrier_evidence.py:225                   f"edge {edge.id}: no first-order saddle log bound to TS "
cloudra/benchmark/barrier_evidence.py:226                   f"{edge.via}; barrier not derivable from evidence"
cloudra/benchmark/barrier_evidence.py:227               )
cloudra/benchmark/barrier_evidence.py:228               continue
cloudra/benchmark/barrier_evidence.py:229           if len(saddles) > 1:
cloudra/benchmark/barrier_evidence.py:230               tokens = [t.lower() for t in (edge.id, edge.via, ts_node_name) if t
]
cloudra/benchmark/barrier_evidence.py:231               preferred = [
cloudra/benchmark/barrier_evidence.py:232                   p for p in saddles if any(t in Path(p).name.lower() for t in
tokens)
cloudra/benchmark/barrier_evidence.py:233               ]
cloudra/benchmark/barrier_evidence.py:234               report.notes.append(
cloudra/benchmark/barrier_evidence.py:235                   f"edge {edge.id}: several saddle logs {saddles}; "
cloudra/benchmark/barrier_evidence.py:236                   + (f"using {preferred[0]} (name matches the step)" if preferred
cloudra/benchmark/barrier_evidence.py:237                      else f"using {saddles[0]} (deterministic first by path)")
cloudra/benchmark/barrier_evidence.py:238               )
cloudra/benchmark/barrier_evidence.py:239               saddles = preferred or saddles
cloudra/benchmark/barrier_evidence.py:240           ts_path = saddles[0]
cloudra/benchmark/barrier_evidence.py:241
cloudra/benchmark/barrier_evidence.py:242           ts_energy, ts_reason = _engine_energy_kcal(sandbox, ts_path)
cloudra/benchmark/barrier_evidence.py:243           base_energy, base_reason = _engine_energy_kcal(sandbox, base_path)
cloudra/benchmark/barrier_evidence.py:244           if ts_energy is None or base_energy is None:
```

```
cloudra/benchmark/barrier_evidence.py:245                report.notes.append(f"edge {edge.id}: {ts_reason or base_reason}")
cloudra/benchmark/barrier_evidence.py:246                continue
cloudra/benchmark/barrier_evidence.py:247            report.steps.append(
cloudra/benchmark/barrier_evidence.py:248                StepBarrier(
cloudra/benchmark/barrier_evidence.py:249                    edge_id=edge.id,
cloudra/benchmark/barrier_evidence.py:250                    ts_path=ts_path,
cloudra/benchmark/barrier_evidence.py:251                    base_path=base_path,
cloudra/benchmark/barrier_evidence.py:252                    base_node_id=edge.from_node,
cloudra/benchmark/barrier_evidence.py:253                    barrier_kcal_mol=ts_energy - base_energy,
cloudra/benchmark/barrier_evidence.py:254                    declared_kcal_mol=edge.barrier_kcal_mol,
cloudra/benchmark/barrier_evidence.py:255                )
cloudra/benchmark/barrier_evidence.py:256            )
cloudra/benchmark/barrier_evidence.py:257
cloudra/benchmark/barrier_evidence.py:258        if not report.steps:
cloudra/benchmark/barrier_evidence.py:259            report.notes.append("no step barrier could be derived from the
delivered logs")
cloudra/benchmark/barrier_evidence.py:260        return report
cloudra/benchmark/barrier_evidence.py:261
cloudra/benchmark/barrier_evidence.py:262
cloudra/benchmark/barrier_evidence.py:263    def load_refusal_declaration(sandbox: Path | str):
cloudra/benchmark/barrier_evidence.py:264        """交付 ``mechanism_graph.json`` 里可选的拒答声明：无则 None（绝不猜）。"""
cloudra/benchmark/barrier_evidence.py:265        import json
cloudra/benchmark/barrier_evidence.py:266
cloudra/benchmark/barrier_evidence.py:267        from cloudra.benchmark.schema import RefusalDeclaration
cloudra/benchmark/barrier_evidence.py:268
cloudra/benchmark/barrier_evidence.py:269        path = Path(sandbox) / "mechanism_graph.json"
cloudra/benchmark/barrier_evidence.py:270        if not path.is_file():
cloudra/benchmark/barrier_evidence.py:271            return None
cloudra/benchmark/barrier_evidence.py:272        try:
cloudra/benchmark/barrier_evidence.py:273            data = json.loads(_read_text(path))
cloudra/benchmark/barrier_evidence.py:274        except ValueError:
cloudra/benchmark/barrier_evidence.py:275            return None
cloudra/benchmark/barrier_evidence.py:276        if not isinstance(data, dict):
cloudra/benchmark/barrier_evidence.py:277            return None
cloudra/benchmark/barrier_evidence.py:278        raw = data.get("refusal")
cloudra/benchmark/barrier_evidence.py:279        if not isinstance(raw, dict):
cloudra/benchmark/barrier_evidence.py:280            return None
cloudra/benchmark/barrier_evidence.py:281        try:
cloudra/benchmark/barrier_evidence.py:282            declaration = RefusalDeclaration.model_validate(raw)
cloudra/benchmark/barrier_evidence.py:283        except Exception:
cloudra/benchmark/barrier_evidence.py:284            return None
cloudra/benchmark/barrier_evidence.py:285        return declaration if declaration.refused else None
cloudra/benchmark/batch.py:1    """Web-hosted benchmark batch controller (GH #32 Phase 1).
cloudra/benchmark/batch.py:2
cloudra/benchmark/batch.py:3    A batch of benchmark tasks runs inside the live RuntimeCore — the same core
cloudra/benchmark/batch.py:4    the API serves. Each task becomes a real project named ``benchmark:{task_id}``
cloudra/benchmark/batch.py:5    whose coordinator works in a sandbox under the benchmark data dir, so the
cloudra/benchmark/batch.py:6    existing project UI (timeline, DagPanel, compute monitor) is the per-task
cloudra/benchmark/batch.py:7    drill-down view for free and the frontend sidebar can group the tasks by the
cloudra/benchmark/batch.py:8    ``benchmark:`` name prefix.
cloudra/benchmark/batch.py:9
cloudra/benchmark/batch.py:10   Differences from the CLI backend (``backend.RuntimeV2Backend``):
cloudra/benchmark/batch.py:11
```

```
cloudra/benchmark/batch.py:12   - No in-process scheduler: submitted compute jobs are claimed by the regular
cloudra/benchmark/batch.py:13     ``cloudra scheduler`` daemon, exactly like live research work.
cloudra/benchmark/batch.py:14   - The method whitelist rides the project settings
cloudra/benchmark/batch.py:15     (``ProjectSettings.method_policy``), enforced by the live pipeline's
cloudra/benchmark/batch.py:16     ``MethodPolicyStage(core=core)`` — no extra stage is appended.
cloudra/benchmark/batch.py:17   - The same ``watch.RunWatcher`` state machine decides when a task has
cloudra/benchmark/batch.py:18     genuinely finished (wait_background / wait_review semantics, GH #30);
cloudra/benchmark/batch.py:19     unattended spawn settings keep ask_user blocked (GH #31).
cloudra/benchmark/batch.py:20   - Scoring reuses ``BenchmarkRunner.score_sandbox``; results land in the same
cloudra/benchmark/batch.py:21     ``benchmark.db`` schema as the CLI runner, tagged with ``batch_id``.
cloudra/benchmark/batch.py:22   """
cloudra/benchmark/batch.py:23
cloudra/benchmark/batch.py:24   from __future__ import annotations
cloudra/benchmark/batch.py:25
cloudra/benchmark/batch.py:26   import asyncio
cloudra/benchmark/batch.py:27   import logging
cloudra/benchmark/batch.py:28   import os
cloudra/benchmark/batch.py:29   import shutil
cloudra/benchmark/batch.py:30   import time
cloudra/benchmark/batch.py:31   from collections import deque
cloudra/benchmark/batch.py:32   from pathlib import Path
cloudra/benchmark/batch.py:33   from typing import Callable, Optional
cloudra/benchmark/batch.py:34
cloudra/benchmark/batch.py:35   from cloudra.benchmark.refusal import RunOutcome, refusal_stats, should_refuse
cloudra/benchmark/batch.py:36   from cloudra.benchmark.runner import (
cloudra/benchmark/batch.py:37       BenchmarkDB,
cloudra/benchmark/batch.py:38       BenchmarkRunner,
cloudra/benchmark/batch.py:39       RunResult,
cloudra/benchmark/batch.py:40       load_tasks,
cloudra/benchmark/batch.py:41   )
cloudra/benchmark/batch.py:42   from cloudra.benchmark.schema import TaskSpec
cloudra/benchmark/batch.py:43   from cloudra.benchmark.watch import RunWatcher
cloudra/benchmark/batch.py:44   from cloudra.runtime_v2.schemas import AgentState, Command, CommandRejected, new_id
cloudra/benchmark/batch.py:45
cloudra/benchmark/batch.py:46   logger = logging.getLogger(__name__)
cloudra/benchmark/batch.py:47
cloudra/benchmark/batch.py:48   # 批次事件流的环形缓冲上限（SSE 断线重连只重放快照，事件缓冲仅服务在线订阅者）。
cloudra/benchmark/batch.py:49   _EVENT_BUFFER_MAX = 500
cloudra/benchmark/batch.py:50
cloudra/benchmark/batch.py:51   _AGENT_TERMINAL = frozenset(
cloudra/benchmark/batch.py:52       {AgentState.SUCCEEDED, AgentState.FAILED, AgentState.CANCELLED}
cloudra/benchmark/batch.py:53   )
cloudra/benchmark/batch.py:54
cloudra/benchmark/batch.py:55
cloudra/benchmark/batch.py:56   def repo_root() -> Path:
cloudra/benchmark/batch.py:57       """仓库根（ideapool 题集与 cloudra 包同仓）。"""
cloudra/benchmark/batch.py:58       import cloudra
cloudra/benchmark/batch.py:59
cloudra/benchmark/batch.py:60       return Path(cloudra.__file__).resolve().parents[1]
cloudra/benchmark/batch.py:61
cloudra/benchmark/batch.py:62
cloudra/benchmark/batch.py:63   def resolve_tasks_dirs(dirs: list[str] | None) -> list[Path]:
```

```
cloudra/benchmark/batch.py:64       """题集目录解析：绝对路径原样；相对路径先按进程 cwd、再按仓库根尝试；
cloudra/benchmark/batch.py:65       缺省回落到仓库内置 easy 档。"""
cloudra/benchmark/batch.py:66       if not dirs:
cloudra/benchmark/batch.py:67           return [repo_root() / "ideapool" / "tasks" / "easy"]
cloudra/benchmark/batch.py:68       out: list[Path] = []
cloudra/benchmark/batch.py:69       for raw in dirs:
cloudra/benchmark/batch.py:70           p = Path(raw)
cloudra/benchmark/batch.py:71           candidates = [p] if p.is_absolute() else [Path.cwd() / p, repo_root() / p]
cloudra/benchmark/batch.py:72           resolved = next((c for c in candidates if c.is_dir()), None)
cloudra/benchmark/batch.py:73           if resolved is None:
cloudra/benchmark/batch.py:74               raise CommandRejected("invalid_request", f"tasks dir not found: {raw}")
cloudra/benchmark/batch.py:75           out.append(resolved)
cloudra/benchmark/batch.py:76       return out
cloudra/benchmark/batch.py:77
cloudra/benchmark/batch.py:78
cloudra/benchmark/batch.py:79   class _TaskRunState:
cloudra/benchmark/batch.py:80       """一题在批次内的运行状态（易失；落库由 benchmark.db 承担）。"""
cloudra/benchmark/batch.py:81
cloudra/benchmark/batch.py:82       def __init__(self, task: TaskSpec) -> None:
cloudra/benchmark/batch.py:83           self.task = task
cloudra/benchmark/batch.py:84           self.status = "queued"  # queued|running|scored|cancelled|error
cloudra/benchmark/batch.py:85           self.project_id = ""
cloudra/benchmark/batch.py:86           self.agent_id = ""
cloudra/benchmark/batch.py:87           self.started_at: Optional[float] = None
cloudra/benchmark/batch.py:88           self.finished_at: Optional[float] = None
cloudra/benchmark/batch.py:89           self.result: Optional[RunResult] = None
cloudra/benchmark/batch.py:90           self.error: str = ""
cloudra/benchmark/batch.py:91
cloudra/benchmark/batch.py:92
cloudra/benchmark/batch.py:93   class BenchmarkBatch:
cloudra/benchmark/batch.py:94       """一批题的异步执行器。同一时间一个服务只持有一个活动批次。"""
cloudra/benchmark/batch.py:95
cloudra/benchmark/batch.py:96       def __init__(
cloudra/benchmark/batch.py:97           self,
cloudra/benchmark/batch.py:98           core,
cloudra/benchmark/batch.py:99           bus,
cloudra/benchmark/batch.py:100          *,
cloudra/benchmark/batch.py:101          data_dir: Path,
cloudra/benchmark/batch.py:102          tasks: list[TaskSpec],
cloudra/benchmark/batch.py:103          tasks_dirs: list[str],
cloudra/benchmark/batch.py:104          concurrency: int,
cloudra/benchmark/batch.py:105          run_timeout_s: float,
cloudra/benchmark/batch.py:106          stuck_grace_s: float,
cloudra/benchmark/batch.py:107          poll_interval_s: float,
cloudra/benchmark/batch.py:108          emit: Callable[[str, dict], None],
cloudra/benchmark/batch.py:109      ) -> None:
cloudra/benchmark/batch.py:110          self.core = core
cloudra/benchmark/batch.py:111          self.bus = bus
cloudra/benchmark/batch.py:112          self.data_dir = Path(data_dir)
cloudra/benchmark/batch.py:113          self.sandboxes = self.data_dir / "sandboxes"
cloudra/benchmark/batch.py:114          self.concurrency = max(1, concurrency)
cloudra/benchmark/batch.py:115          self.emit = emit
```